L-BFGS-B

3.0

Generated by Doxygen 1.9.1

1 L-BFGS-B 1

2 File Index 3

2.1 File List . . . 3

3 File Documentation 4

3.1 src/active.f File Reference . . . 4

3.1.1 Function/Subroutine Documentation . . . 4

3.2 src/bmv.f File Reference . . . 5

3.2.1 Function/Subroutine Documentation . . . 5

3.3 src/cauchy.f File Reference . . . 6

3.3.1 Function/Subroutine Documentation . . . 6

3.4 src/cmprlb.f File Reference . . . 10

3.4.1 Function/Subroutine Documentation . . . 10

3.5 src/dcsrch.f File Reference . . . 11

3.5.1 Function/Subroutine Documentation . . . 11

3.6 src/dcstep.f File Reference . . . 13

3.6.1 Function/Subroutine Documentation . . . 14

3.7 src/errclb.f File Reference . . . 15

3.7.1 Function/Subroutine Documentation . . . 15

3.8 src/formk.f File Reference . . . 16

3.8.1 Function/Subroutine Documentation . . . 16

3.9 src/formt.f File Reference . . . 18

3.9.1 Function/Subroutine Documentation . . . 18

3.10 src/freev.f File Reference . . . 19

3.10.1 Function/Subroutine Documentation . . . 19

3.11 src/hpsolb.f File Reference . . . 20

3.11.1 Function/Subroutine Documentation . . . 20

3.12 src/lnsrlb.f File Reference . . . 21

3.12.1 Function/Subroutine Documentation . . . 21

3.13 src/mainlb.f File Reference . . . 24

3.13.1 Function/Subroutine Documentation . . . 24

3.14 src/matupd.f File Reference . . . 28

3.14.1 Function/Subroutine Documentation . . . 28

3.15 src/prn1lb.f File Reference . . . 29

3.15.1 Function/Subroutine Documentation . . . 29

3.16 src/prn2lb.f File Reference . . . 31

3.16.1 Function/Subroutine Documentation . . . 31

3.17 src/prn3lb.f File Reference . . . 32

3.17.1 Function/Subroutine Documentation . . . 32

3.18 src/projgr.f File Reference . . . 34

3.18.1 Function/Subroutine Documentation . . . 34

3.19 src/setulb.f File Reference . . . 35

3.19.1 Function/Subroutine Documentation . . . 35
3.20 src/subsm.f File Reference . . . 39
3.20.1 Function/Subroutine Documentation . . . 39
3.21 src/timer.f File Reference . . . 42
3.21.1 Function/Subroutine Documentation . . . 42

4 Example Documentation **43**
4.1 driver1.f . . . 43
4.2 driver1.f90 . . . 47
4.3 driver2.f . . . 51
4.4 driver2.f90 . . . 54
4.5 driver3.f . . . 57
4.6 driver3.f90 . . . 60

Index **65**

1 L-BFGS-B

Software for Large-scale Bound-constrained Optimization

L-BFGS-B is a limited-memory quasi-Newton code for bound-constrained optimization, i.e., for problems where the only constraints are of the form `l <= x <= u`. It is intended for problems in which information on the Hessian matrix is difficult to obtain, or for large dense problems. L-BFGS-B can also be used for unconstrained problems, and in this case performs similarly to its predecessor, algorithm L-BFGS (Harwell routine VA15). The algorithm is implemented in Fortran 77.

Authors

- Ciyou Zhu
- Richard Byrd
- Jorge Nocedal
- Jose Luis Morales

Related Publications

- R. H. Byrd, P. Lu, J. Nocedal and C. Zhu. A Limited Memory Algorithm for Bound Constrained Optimization (1995), SIAM Journal on Scientific and Statistical Computing, Vol. 16, Num. 5, pp. 1190-1208
- C. Zhu, R. H. Byrd and J. Nocedal. L-BFGS-B: Algorithm 778: L-BFGS-B, FORTRAN routines for large scale bound constrained optimization (1997), ACM Transactions on Mathematical Software, Vol. 23, Num. 4, pp. 550-560
- J.L. Morales and J. Nocedal. L-BFGS-B: Remark on Algorithm 778: L-BFGS-B, FORTRAN routines for large scale bound constrained optimization (2011), ACM Transactions on Mathematical Software, Vol. 38, Num. 1

- R. H. Byrd, J. Nocedal and R. B. Schnabel. Representations of quasi-Newton matrices and their use in limited memory methods (1994), Mathematical Programming, Vol. 63, pp. 129-156

Note that the subspace minimization in the LBFGSpp implementation is an exact minimization subject to the bounds, based on the BOXCQP algorithm:

- C. Voglis and I. E. Lagaris, BOXCQP: An Algorithm for Bound Constrained Convex Quadratic Problems (2004), 1st International Conference "From Scientific Computing to Computational Engineering", Athens, Greece

For an eagle-eye overview of L-BFGS-B and the genealogy BFGS->L-BFGS->L-BFGS-B, see Henao's Master's thesis.

Related Software

- wilmerhenao/L-BFGS-B-NS: An L-BFGS-B-NS Optimizer for Non-Smooth Functions
- pcarbo/lbfgsb-matlab: A MATLAB interface for L-BFGS-B
- bgranzow/L-BFGS-B: A pure Matlab implementation of L-BFGS-B (LBFGSB)
- constantino-garcia/lbfgsb_cpp_wrapper: A simple C++ wrapper around the original Fortran L-BGSG-B routine
- yixuan/LBFGSpp: A header-only C++ library for L-BFGS and L-BFGS-B algorithms
- chokkan/liblbfgs: libLBFGS: a library of Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)
- mkobos/lbfgsb_wrapper: Java wrapper for the Fortran L-BFGS-B algorithm
- yuhonglin/Lbfgsb.jl: A Julia wrapper of the l-bfgs-b fortran library
- Gnimuc/LBFGSB.jl: Julia wrapper for L-BFGS-B Nonlinear Optimization Code
- afbarnard/go-lbfgsb: L-BFGS-B optimization for Go, C, Fortran 2003
- nepluno/lbfgsb-gpu: An open source library for the GPU-implementation of L-BFGS-B algorithm
- Chris00/L-BFGS-ocaml: OCaml bindings for L-BFGS
- dwicke/L-BFGS-B-Lua: L-BFGS-B lua wrapper around a L-BFGS-B C implementation
- avieira/python_lbfgsb: Pure Python-based L-BFGS-B implementation
- ybyygu/rust-lbfgsb: Ergonomic bindings to L-BFGS-B code for Rust
- rforge/lbfgsb3c: Limited Memory BFGS Minimizer with Bounds on Parameters with optim() 'C' Interface for R
- florafauna/optimParallel-python: A parallel version of 'scipy.optimize.minimize(method='L-BFGS-B')`

Notes on this repository

I (J. Schilling) took the freedom to

- put the L-BFGS-B code obtained from the original website up in this repository,
- divide the subroutines and functions into separate files,
- convert parts of the documentation into a format understandable to doxygen and
- replace the included BLAS/LINPACK routines with calls to user-provided BLAS/LAPACK routines. To be precise, the calls to LINPACK's dtrsl were replace with calls to LAPACK's dtrsm and the calls to LINPACK's dpofa were replaces with calls to LAPACK's dpotrf.

Building

A CMake setup is provided for L-BFGS-B in this repository. External modules for BLAS and LAPACK have to be installed on your system. Then, building the shared library liblbfgsb.so and the examples driver*.f and driver*.f90 works as follows:

```
> mkdir build
> cd build
> cmake ..
> make -j
```

The resulting shared library and the driver executables can be found in the build directory.

Concluding remarks

The current release is version 3.0. The distribution file was last changed on 02/08/11.

This work was in no way intending to infringe any copyrights or take credit for others' work. Feel free to contact me at any time in case you noticed something against the rules. Above documentation is obtained from the archived version of the original manual.

A PDF version of the documentation is available here: L-BFGS-B.pdf.

2 File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/active.f 4

src/bmv.f 5

src/cauchy.f 6

src/cmprlb.f 10

src/dcsrch.f 11

src/dcstep.f 13

src/errclb.f 15

src/formk.f 16

src/formt.f 18

src/freev.f 19

src/hpsolb.f 20

src/lnsrlb.f 21

src/mainlb.f 24

src/matupd.f 28

src/prn1lb.f 29

src/prn2lb.f 31

src/prn3lb.f 32

src/projgr.f 34

src/setulb.f 35

src/subsm.f 39

src/timer.f 42

3 File Documentation

3.1 src/active.f File Reference

Functions/Subroutines

- subroutine active (n, l, u, nbd, x, iwhere, iprint, prjctd, cnstnd, boxed)

  This subroutine initializes iwhere and projects the initial x to the feasible set if necessary.

3.1.1 Function/Subroutine Documentation

3.1.1.1 active()

```
subroutine active (
        integer n,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision, dimension(n) x,
        integer, dimension(n) iwhere,
        integer iprint,
        logical prjctd,
        logical cnstnd,
        logical boxed )
```

This subroutine initializes iwhere and projects the initial x to the feasible set if necessary.

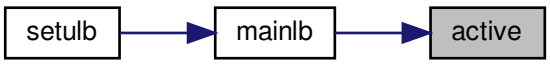
Parameters

n	number of parameters
l	lower bounds on parameters
u	upper bounds on parameters
nbd	indicates which bounds are present
x	position
iwhere	On entry iwhere is unspecified. On exit: iwhere(i)= • -1 if x(i) has no bounds • 3 if l(i)=u(i), • 0 otherwise. In cauchy, iwhere is given finer gradations.
iprint	console output flag
prjctd	TODO
cnstnd	TODO
boxed	TODO

Definition at line 32 of file active.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.2 src/bmv.f File Reference

Functions/Subroutines

- subroutine bmv (m, sy, wt, col, v, p, info)

  This subroutine computes the product of the 2m x 2m middle matrix in the compact L-BFGS formula of B and a 2m vector v.

3.2.1 Function/Subroutine Documentation

3.2.1.1 bmv()

```
subroutine bmv (
        integer m,
        double precision, dimension(m, m) sy,
        double precision, dimension(m, m) wt,
        integer col,
        double precision, dimension(2*col) v,
        double precision, dimension(2*col) p,
        integer info )
```

This subroutine computes the product of the 2m x 2m middle matrix in the compact L-BFGS formula of B and a 2m vector v; it returns the product in p.

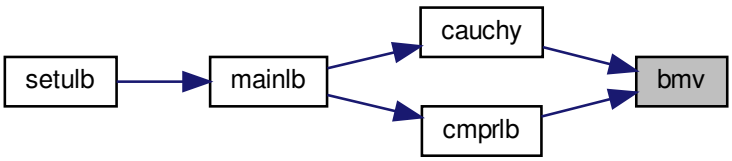
Parameters

m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
sy	On entry sy specifies the matrix S'Y. On exit sy is unchanged.
wt	On entry wt specifies the upper triangular matrix J' which is the Cholesky factor of (thetaS'S+LD^(-1)L'). On exit wt is unchanged.
col	On entry col specifies the number of s-vectors (or y-vectors) stored in the compact L-BFGS formula. On exit col is unchanged.
v	On entry v specifies vector v. On exit v is unchanged.
p	On entry p is unspecified. On exit p is the product Mv.
info	On entry info is unspecified. On exit info = • 0 for normal return, • nonzero for abnormal return when the system to be solved by dtrsl is singular.

Definition at line 35 of file bmv.f.

Referenced by cauchy(), and cmprlb().

Here is the caller graph for this function:



3.3 src/cauchy.f File Reference

Functions/Subroutines

- subroutine cauchy (n, x, l, u, nbd, g, iorder, iwhere, t, d, xcp, m, wy, ws, sy, wt, theta, col, head, p, c, wbp, v, nseg, iprint, sbgnrm, info, epsmch)

  Compute the Generalized Cauchy Point along the projected gradient direction.

3.3.1 Function/Subroutine Documentation

3.3.1.1 cauchy() `subroutine cauchy (`

```
integer n,
double precision, dimension(n) x,
double precision, dimension(n) l,
double precision, dimension(n) u,
integer, dimension(n) nbd,
double precision, dimension(n) g,
integer, dimension(n) iorder,
integer, dimension(n) iwhere,
double precision, dimension(n) t,
double precision, dimension(n) d,
double precision, dimension(n) xcp,
integer m,
double precision, dimension(n, col) wy,
double precision, dimension(n, col) ws,
double precision, dimension(m, m) sy,
double precision, dimension(m, m) wt,
double precision theta,
integer col,
integer head,
double precision, dimension(2*m) p,
double precision, dimension(2*m) c,
double precision, dimension(2*m) wbp,
double precision, dimension(2*m) v,
integer nseg,
integer iprint,
double precision sbgnrm,
integer info,
double precision epsmch )
```

For given x, l, u, g (with sbgnrm $>$ 0), and a limited memory BFGS matrix B defined in terms of matrices WY, WS, WT, and scalars head, col, and theta, this subroutine computes the generalized Cauchy point (GCP), defined as the first local minimizer of the quadratic

```
Q(x + s) = g's + 1/2 s'Bs
```

along the projected gradient direction P(x-tg,l,u). The routine returns the GCP in xcp.

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.

Parameters

x	On entry x is the starting point for the GCP computation. On exit x is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
g	On entry g is the gradient of f(x). g must be a nonzero vector. On exit g is unchanged.
iorder	iorder will be used to store the breakpoints in the piecewise linear path and free variables encountered. On exit, • iorder(1),...,iorder(nleft) are indices of breakpoints which have not been encountered; • iorder(nleft+1),...,iorder(nbreak) are indices of encountered breakpoints; and • iorder(nfree),...,iorder(n) are indices of variables which have no bound constraits along the search direction.
iwhere	On entry iwhere indicates only the permanently fixed (iwhere=3) or free (iwhere= -1) components of x. On exit iwhere records the status of the current x variables. iwhere(i)= • -3 if x(i) is free and has bounds, but is not moved • 0 if x(i) is free and has bounds, and is moved • 1 if x(i) is fixed at l(i), and l(i) .ne. u(i) • 2 if x(i) is fixed at u(i), and u(i) .ne. l(i) • 3 if x(i) is always fixed, i.e., u(i)=x(i)=l(i) • -1 if x(i) is always free, i.e., it has no bounds.
t	working array; will be used to store the break points.
d	the Cauchy direction P(x-tg)-x
xcp	returns the GCP on exit
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.

Parameters

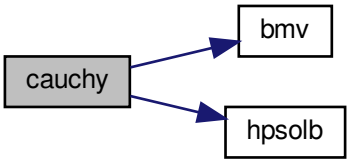
wt	On entry this stores the Cholesky factorization of (theta∗S'S+LD^(-1)L'), that defines the limited memory BFGS matrix. On exit this array is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
p	will be used to store the vector p = W^(T)d.
c	will be used to store the vector c = W^(T)(xcp-x).
wbp	will be used to store the row of W corresponding to a breakpoint.
v	working array
nseg	On exit nseg records the number of quadratic segments explored in searching for the GCP.
iprint	variable that must be set by the user. It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
sbgnrm	On entry sbgnrm is the norm of the projected gradient at x. On exit sbgnrm is unchanged.
info	On entry info is 0. On exit info = • 0 for normal return, • = nonzero for abnormal return when the the system used in routine bmv is singular.
epsmch	machine precision epsilon

Definition at line 130 of file cauchy.f.

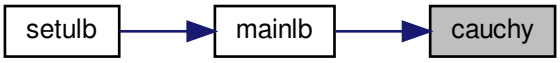
References bmv(), and hpsolb().

Referenced by mainlb().

Here is the call graph for this function:



Here is the caller graph for this function:



3.4 src/cmprlb.f File Reference

Functions/Subroutines

- subroutine cmprlb (n, m, x, g, ws, wy, sy, wt, z, r, wa, index, theta, col, head, nfree, cnstnd, info)

  This subroutine computes r=-Z'B(xcp-xk)-Z'g by using wa(2m+1)=W'(xcp-x) from subroutine cauchy.

3.4.1 Function/Subroutine Documentation

3.4.1.1 cmprlb()

```
subroutine cmprlb (
        integer n,
        integer m,
        double precision, dimension(n) x,
        double precision, dimension(n) g,
        double precision, dimension(n, m) ws,
        double precision, dimension(n, m) wy,
        double precision, dimension(m, m) sy,
        double precision, dimension(m, m) wt,
        double precision, dimension(n) z,
        double precision, dimension(n) r,
        double precision, dimension(4*m) wa,
        integer, dimension(n) index,
```

```
double precision theta,
integer col,
integer head,
integer nfree,
logical cnstnd,
integer info )
```

This subroutine computes r=-Z'B(xcp-xk)-Z'g by using wa(2m+1)=W'(xcp-x) from subroutine cauchy.

Parameters

n	number of parameters
m	history size of Hessian approximation
x	position
g	gradient
ws	part of L-BFGS matrix
wy	part of L-BFGS matrix
sy	part of L-BFGS matrix
wt	part of L-BFGS matrix
z	TODO
r	TODO
wa	TODO
index	TODO
theta	TODO
col	TODO
head	TODO
nfree	TODO
cnstnd	TODO
info	TODO

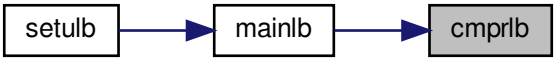
Definition at line 27 of file cmprlb.f.

References bmv().

Referenced by mainlb().

Here is the call graph for this function:

Here is the caller graph for this function:



3.5 src/dcsrch.f File Reference

Functions/Subroutines

- subroutine dcsrch (f, g, stp, ftol, gtol, xtol, stpmin, stpmax, task, isave, dsave)

  This subroutine finds a step that satisfies a sufficient decrease condition and a curvature condition.

3.5.1 Function/Subroutine Documentation

3.5.1.1 dcsrch() subroutine dcsrch (

```
double precision f,
double precision g,
double precision stp,
double precision ftol,
double precision gtol,
double precision xtol,
double precision stpmin,
double precision stpmax,
character*(*) task,
integer, dimension(2) isave,
double precision, dimension(13) dsave )
```

This subroutine finds a step that satisfies a sufficient decrease condition and a curvature condition.

Each call of the subroutine updates an interval with endpoints stx and sty. The interval is initially chosen so that it contains a minimizer of the modified function

```
psi(stp) = f(stp) - f(0) - ftol*stp*f'(0).
```

If psi(stp) <= 0 and f'(stp) >= 0 for some step, then the interval is chosen so that it contains a minimizer of f.

The algorithm is designed to find a step that satisfies the sufficient decrease condition

```
f(stp) <= f(0) + ftol*stp*f'(0),
```

and the curvature condition

```
abs(f'(stp)) <= gtol*abs(f'(0)).
```

If ftol is less than gtol and if, for example, the function is bounded below, then there is always a step which satisfies both conditions.

If no step can be found that satisfies both conditions, then the algorithm stops with a warning. In this case stp only satisfies the sufficient decrease condition.

A typical invocation of dcsrch has the following outline:

```
   task = 'START'
10 continue
   call dcsrch( ... )
   if (task .eq. 'FG') then
     evaluate the function and the gradient at stp
   goto 10
   end if
```

NOTE: The user must no alter work arrays between calls.

Parameters

f	On initial entry f is the value of the function at 0. On subsequent entries f is the value of the function at stp. On exit f is the value of the function at stp.
g	On initial entry g is the derivative of the function at 0. On subsequent entries g is the derivative of the function at stp. On exit g is the derivative of the function at stp.
stp	On entry stp is the current estimate of a satisfactory step. On initial entry, a positive initial estimate must be provided. On exit stp is the current estimate of a satisfactory step if task = 'FG'. If task = 'CONV' then stp satisfies the sufficient decrease and curvature condition.
ftol	On entry ftol specifies a nonnegative tolerance for the sufficient decrease condition. On exit ftol is unchanged.
gtol	On entry gtol specifies a nonnegative tolerance for the curvature condition. On exit gtol is unchanged.
xtol	On entry xtol specifies a nonnegative relative tolerance for an acceptable step. The subroutine exits with a warning if the relative difference between sty and stx is less than xtol. On exit xtol is unchanged.
stpmin	On entry stpmin is a nonnegative lower bound for the step. On exit stpmin is unchanged.
stpmax	On entry stpmax is a nonnegative upper bound for the step. On exit stpmax is unchanged.
task	On initial entry task must be set to 'START'. On exit task indicates the required action: • If task(1:2) = 'FG' then evaluate the function and derivative at stp and call dcsrch again. • If task(1:4) = 'CONV' then the search is successful. • If task(1:4) = 'WARN' then the subroutine is not able to satisfy the convergence conditions. The exit value of stp contains the best point found during the search. • If task(1:5) = 'ERROR' then there is an error in the input arguments. On exit with convergence, a warning or an error, the variable task contains additional information.
isave	work array
dsave	work array

Definition at line 94 of file dcsrch.f.

References dcstep().

Referenced by lnsrlb().

Here is the call graph for this function:

Here is the caller graph for this function:

3.6 src/dcstep.f File Reference

Functions/Subroutines

- subroutine dcstep (stx, fx, dx, sty, fy, dy, stp, fp, dp, brackt, stpmin, stpmax)

  This subroutine computes a safeguarded step for a search procedure and updates an interval that contains a step that satisfies a sufficient decrease and a curvature condition.

3.6.1 Function/Subroutine Documentation

3.6.1.1 dcstep()

```
subroutine dcstep (
        double precision stx,
        double precision fx,
        double precision dx,
        double precision sty,
        double precision fy,
        double precision dy,
        double precision stp,
        double precision fp,
        double precision dp,
        logical brackt,
        double precision stpmin,
        double precision stpmax )
```

This subroutine computes a safeguarded step for a search procedure and updates an interval that contains a step that satisfies a sufficient decrease and a curvature condition.

The parameter stx contains the step with the least function value. If brackt is set to .true. then a minimizer has been bracketed in an interval with endpoints stx and sty. The parameter stp contains the current step. The subroutine assumes that if brackt is set to .true. then

```
min(stx,sty) < stp < max(stx,sty),
```

and that the derivative at stx is negative in the direction of the step.

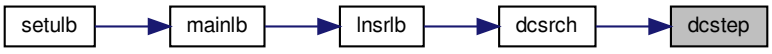
Parameters

stx	On entry stx is the best step obtained so far and is an endpoint of the interval that contains the minimizer. On exit stx is the updated best step.
fx	On entry fx is the function at stx. On exit fx is the function at stx.
dx	On entry dx is the derivative of the function at stx. The derivative must be negative in the direction of the step, that is, dx and stp - stx must have opposite signs. On exit dx is the derivative of the function at stx.
sty	On entry sty is the second endpoint of the interval that contains the minimizer. On exit sty is the updated endpoint of the interval that contains the minimizer.
fy	On entry fy is the function at sty. On exit fy is the function at sty.
dy	On entry dy is the derivative of the function at sty. On exit dy is the derivative of the function at the exit sty.
stp	On entry stp is the current step. If brackt is set to .true. then on input stp must be between stx and sty. On exit stp is a new trial step.
fp	On entry fp is the function at stp. On exit fp is unchanged.
dp	On entry dp is the the derivative of the function at stp. On exit dp is unchanged.
brackt	On entry brackt specifies if a minimizer has been bracketed. Initially brackt must be set to .false. On exit brackt specifies if a minimizer has been bracketed. When a minimizer is bracketed brackt is set to .true.
stpmin	On entry stpmin is a lower bound for the step. On exit stpmin is unchanged.
stpmax	On entry stpmax is an upper bound for the step. On exit stpmax is unchanged.

Definition at line 66 of file dcstep.f.

Referenced by dcsrch().

Here is the caller graph for this function:



3.7 src/errclb.f File Reference

Functions/Subroutines

- subroutine errclb (n, m, factr, l, u, nbd, task, info, k)

  This subroutine checks the validity of the input data.

3.7.1 Function/Subroutine Documentation

3.7.1.1 errclb() `subroutine errclb (`

```
integer n,
integer m,
double precision factr,
double precision, dimension(n) l,
double precision, dimension(n) u,
integer, dimension(n) nbd,
character*60 task,
integer info,
integer k )
```

This subroutine checks the validity of the input data.

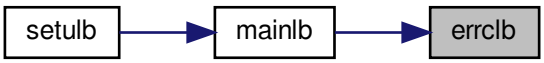
Parameters

n	number of parameters
m	history size of approximated Hessian
factr	convergence criterion on function value
l	lower bounds for parameters
u	upper bounds for parameters
nbd	indicates which bounds are present
task	if an error occurs, contains a human-readable error message
info	=0 on success; =-6 if nbd(k) was invalid; =-7 if both limits are given but l(k) $>$ u(k)
k	index of last errournous parameter

Definition at line 16 of file errclb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.8 src/formk.f File Reference

Functions/Subroutines

- subroutine formk (n, nsub, ind, nenter, ileave, indx2, iupdat, updatd, wn, wn1, m, ws, wy, sy, theta, col, head, info)

  Forms the LEL^T factorization of the indefinite matrix K.

3.8.1 Function/Subroutine Documentation

3.8.1.1 formk()

```
subroutine formk (
        integer n,
        integer nsub,
        integer, dimension(n) ind,
        integer nenter,
        integer ileave,
        integer, dimension(n) indx2,
        integer iupdat,
        logical updatd,
        double precision, dimension(2*m, 2*m) wn,
        double precision, dimension(2*m, 2*m) wn1,
        integer m,
        double precision, dimension(n, m) ws,
        double precision, dimension(n, m) wy,
        double precision, dimension(m, m) sy,
        double precision theta,
        integer col,
        integer head,
        integer info )
```

This subroutine forms the LEL^T factorization of the indefinite matrix

K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta*S'AA'S] where E = [-I 0] [0 I]

The matrix K can be shown to be equal to the matrix M^[-1]N occurring in section 5.1 of [1], as well as to the matrix Mbar^[-1] Nbar in section 5.3.

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
nsub	On entry nsub is the number of subspace variables in free set. On exit nsub is not changed.
ind	On entry ind specifies the indices of subspace variables. On exit ind is unchanged.
nenter	On entry nenter is the number of variables entering the free set. On exit nenter is unchanged.
ileave	On entry indx2(ileave),...,indx2(n) are the variables leaving the free set. On exit ileave is unchanged.
indx2	On entry indx2(1),...,indx2(nenter) are the variables entering the free set, while indx2(ileave),...,indx2(n) are the variables leaving the free set. On exit indx2 is unchanged.
iupdat	On entry iupdat is the total number of BFGS updates made so far. On exit iupdat is unchanged.
updatd	On entry 'updatd' is true if the L-BFGS matrix is updated. On exit 'updatd' is unchanged.
wn	On entry wn is unspecified. On exit the upper triangle of wn stores the LEL^T factorization of the 2*col x 2*col indefinite matrix [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta*S'AA'S]

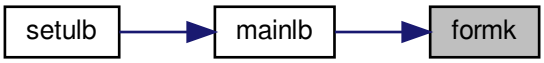
Parameters

wn1	On entry wn1 stores the lower triangular part of [Y' ZZ'Y L_a'+R_z'] [L_a+R_z S'AA'S] in the previous iteration. On exit wn1 stores the corresponding updated matrices. The purpose of wn1 is just to store these inner products so they can be easily updated and inserted into wn.
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
info	On entry info is unspecified. On exit info • = 0 for normal return; • = -1 when the 1st Cholesky factorization failed; • = -2 when the 2st Cholesky factorization failed.

Definition at line 89 of file formk.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.9 src/formt.f File Reference

Functions/Subroutines

- subroutine formt (m, wt, sy, ss, col, theta, info)

  Forms the upper half of the pos. def. and symm. T.

3.9.1 Function/Subroutine Documentation

3.9.1.1 formt()

```
subroutine formt (
        integer m,
        double precision, dimension(m, m) wt,
        double precision, dimension(m, m) sy,
        double precision, dimension(m, m) ss,
        integer col,
        double precision theta,
        integer info )
```

This subroutine forms the upper half of the pos. def. and symm. T = theta∗SS + L∗D^(-1)∗L', stores T in the upper triangle of the array wt, and performs the Cholesky factorization of T to produce J∗J', with J' stored in the upper triangle of wt.

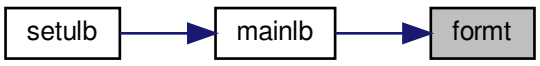
Parameters

m	history size of approximated Hessian
wt	part of L-BFGS matrix
sy	part of L-BFGS matrix
ss	part of L-BFGS matrix
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
info	error/success indicator

Definition at line 21 of file formt.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.10 src/freev.f File Reference

Functions/Subroutines

- subroutine freev (n, nfree, index, nenter, ileave, indx2, iwhere, wrk, updatd, cnstnd, iprint, iter)

  This subroutine counts the entering and leaving variables when iter > 0, and finds the index set of free and active variables at the GCP.

3.10.1 Function/Subroutine Documentation

3.10.1.1 freev()

```
subroutine freev (
        integer n,
        integer nfree,
        integer, dimension(n) index,
        integer nenter,
        integer ileave,
        integer, dimension(n) indx2,
        integer, dimension(n) iwhere,
        logical wrk,
        logical updatd,
        logical cnstnd,
        integer iprint,
        integer iter )
```

This subroutine counts the entering and leaving variables when iter $>$ 0, and finds the index set of free and active variables at the GCP.

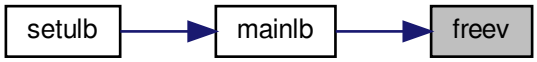
Parameters

n	number of parameters
nfree	number of free parameters, i.e., those not at their bounds
index	for i=1,...,nfree, index(i) are the indices of free variables for i=nfree+1,...,n, index(i) are the indices of bound variables On entry after the first iteration, index gives the free variables at the previous iteration. On exit it gives the free variables based on the determination in cauchy using the array iwhere.
nenter	TODO
ileave	TODO
indx2	On entry indx2 is unspecified. On exit with iter$>$0, indx2 indicates which variables have changed status since the previous iteration. For i= 1,...,nenter, indx2(i) have changed from bound to free. For i= ileave+1,...,n, indx2(i) have changed from free to bound.
iwhere	TODO
wrk	TODO
updatd	TODO
cnstnd	indicating whether bounds are present
iprint	control screen output
iter	TODO

Definition at line 32 of file freev.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.11 src/hpsolb.f File Reference

Functions/Subroutines

- subroutine hpsolb (n, t, iorder, iheap)

  This subroutine sorts out the least element of t, and puts the remaining elements of t in a heap.

3.11.1 Function/Subroutine Documentation

3.11.1.1 hpsolb()

```
subroutine hpsolb (
    integer n,
    double precision, dimension(n) t,
    integer, dimension(n) iorder,
    integer iheap )
```

Parameters

n	On entry n is the dimension of the arrays t and iorder. On exit n is unchanged.
t	On entry t stores the elements to be sorted. On exit t(n) stores the least elements of t, and t(1) to t(n-1) stores the remaining elements in the form of a heap.
iorder	On entry iorder(i) is the index of t(i). On exit iorder(i) is still the index of t(i), but iorder may be permuted in accordance with t.
iheap	On entry iheap should be set as follows: • iheap .eq. 0 if t(1) to t(n) is not in the form of a heap, • iheap .ne. 0 if otherwise. On exit iheap is unchanged.

Definition at line 21 of file hpsolb.f.

Referenced by cauchy().

Here is the caller graph for this function:

3.12 src/lnsrlb.f File Reference

Functions/Subroutines

- subroutine lnsrlb (n, l, u, nbd, x, f, fold, gd, gdold, g, d, r, t, z, stp, dnorm, dtd, xstep, stpmx, iter, ifun, iback, nfgv, info, task, boxed, cnstnd, csave, isave, dsave)

  This subroutine calls subroutine dcsrch from the Minpack2 library to perform the line search. Subroutine dscrch is safeguarded so that all trial points lie within the feasible region.

3.12.1 Function/Subroutine Documentation

3.12.1.1 lnsrlb()

```
subroutine lnsrlb (
        integer n,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision, dimension(n) x,
        double precision f,
        double precision fold,
        double precision gd,
        double precision gdold,
        double precision, dimension(n) g,
        double precision, dimension(n) d,
        double precision, dimension(n) r,
        double precision, dimension(n) t,
        double precision, dimension(n) z,
        double precision stp,
        double precision dnorm,
        double precision dtd,
        double precision xstep,
        double precision stpmx,
        integer iter,
        integer ifun,
        integer iback,
        integer nfgv,
        integer info,
        character*60 task,
        logical boxed,
        logical cnstnd,
        character*60 csave,
        integer, dimension(2) isave,
        double precision, dimension(13) dsave )
```

Parameters

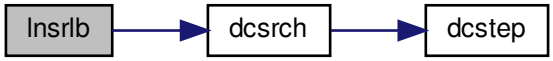
n	number of parameters
l	lower bounds of parameters
u	upper bounds of parameters
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
x	position
f	function value at x
fold	TODO
gd	TODO
gdold	TODO
g	gradient of f at x
d	TODO
r	TODO
t	TODO
z	TODO
stp	TODO
dnorm	TODO
dtd	TODO
xstep	TODO
stpmx	TODO
iter	TODO
ifun	TODO
iback	TODO
nfgv	TODO
info	TODO
task	TODO
boxed	TODO
cnstnd	TODO
csave	working array
isave	working array
dsave	working array

Definition at line 43 of file lnsrlb.f.

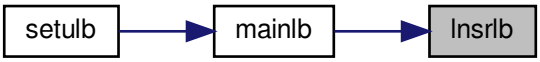
References dcsrch().

Referenced by mainlb().

Here is the call graph for this function:



Here is the caller graph for this function:



3.13 src/mainlb.f File Reference

Functions/Subroutines

- subroutine mainlb (n, m, x, l, u, nbd, f, g, factr, pgtol, ws, wy, sy, ss, wt, wn, snd, z, r, d, t, xp, wa, index, iwhere, indx2, task, iprint, csave, lsave, isave, dsave)

  This subroutine solves bound constrained optimization problems by using the compact formula of the limited memory BFGS updates.

3.13.1 Function/Subroutine Documentation

3.13.1.1 mainlb()

```
subroutine mainlb (
        integer n,
        integer m,
        double precision, dimension(n) x,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision f,
        double precision, dimension(n) g,
        double precision factr,
        double precision pgtol,
        double precision, dimension(n, m) ws,
        double precision, dimension(n, m) wy,
        double precision, dimension(m, m) sy,
        double precision, dimension(m, m) ss,
```

```
double precision, dimension(m, m) wt,
double precision, dimension(2*m, 2*m) wn,
double precision, dimension(2*m, 2*m) snd,
double precision, dimension(n) z,
double precision, dimension(n) r,
double precision, dimension(n) d,
double precision, dimension(n) t,
double precision, dimension(n) xp,
double precision, dimension(8*m) wa,
integer, dimension(n) index,
integer, dimension(n) iwhere,
integer, dimension(n) indx2,
character*60 task,
integer iprint,
character*60 csave,
logical, dimension(4) lsave,
integer, dimension(23) isave,
double precision, dimension(29) dsave )
```

This subroutine solves bound constrained optimization problems by using the compact formula of the limited memory BFGS updates.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit m is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
g	On first entry g is unspecified. On final exit g is the value of the gradient at x.
factr	On entry factr >= 0 is specified by the user. The iteration will stop when (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch where epsmch is the machine precision, which is automatically generated by the code. On exit factr is unchanged.
pgtol	On entry pgtol >= 0 is specified by the user. The iteration will stop when max{|proj g_i | i = 1, ..., n} <= pgtol where pg_i is the ith component of the projected gradient. On exit pgtol is unchanged.

Parameters

ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
ss	On entry this stores S'S, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wt	On entry this stores the Cholesky factorization of (theta∗S'S+LD^(-1)L'), that defines the limited memory BFGS matrix. See eq. (2.26) in [3]. On exit this array is unchanged.
wn	working array used to store the LEL^T factorization of the indefinite matrix K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta∗S'AA'S] where E = [-I 0] [0 I]
snd	working array used to store the lower triangular part of N = [Y' ZZ'Y L_a'+R_z'] [L_a +R_z S'AA'S]
z	working array used at different times to store the Cauchy point and the Newton point.
r	working array
d	working array
t	working array
xp	working array used to safeguard the projected Newton direction
wa	working array
index	In subroutine freev, index is used to store the free and fixed variables at the Generalized Cauchy Point (GCP).
iwhere	working array used to record the status of the vector x for GCP computation. iwhere(i)= • 0 or -3 if x(i) is free and has bounds, • 1 if x(i) is fixed at l(i), and l(i) .ne. u(i) • 2 if x(i) is fixed at u(i), and u(i) .ne. l(i) • 3 if x(i) is always fixed, i.e., u(i)=x(i)=l(i) • -1 if x(i) is always free, i.e., no bounds on it.
indx2	working array Within subroutine cauchy, indx2 corresponds to the array iorder. In subroutine freev, a list of variables entering and leaving the free set is stored in indx2, and it is passed on to subroutine formk with this information.
task	working string indicating the current job when entering and leaving this subroutine.
iprint	It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
csave	working string

Parameters

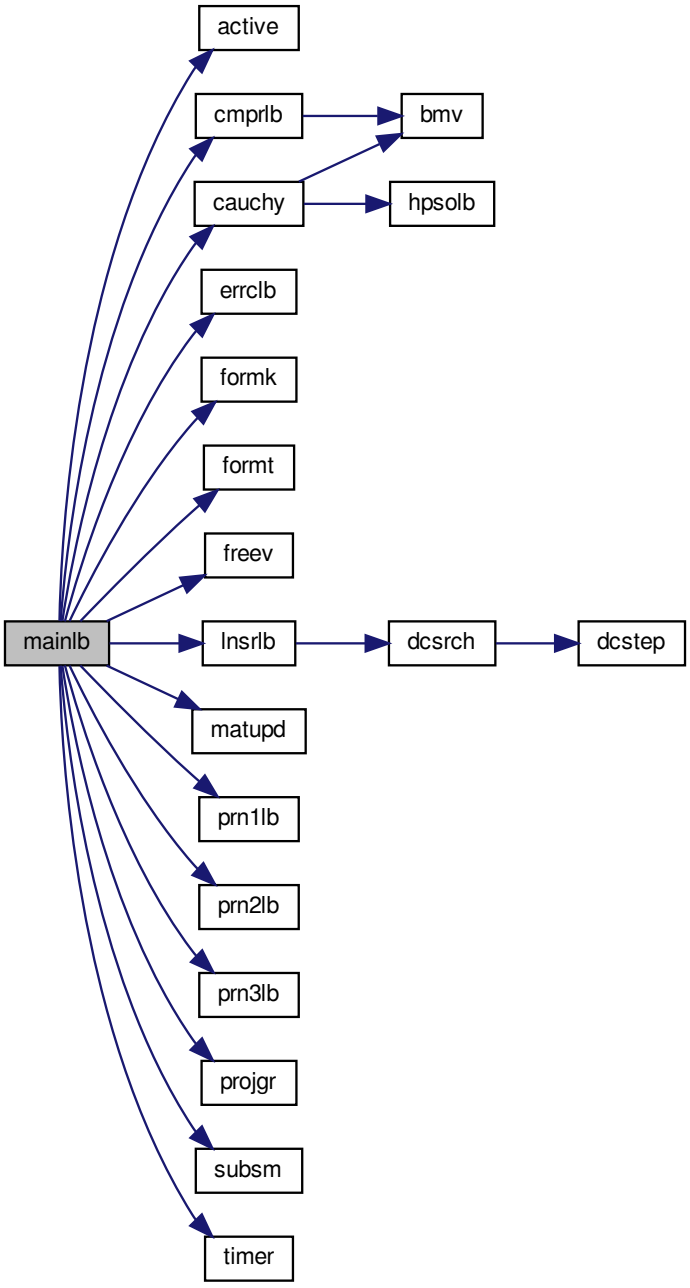
lsave	working array
isave	working array
dsave	working array

Definition at line 128 of file mainlb.f.

References active(), cauchy(), cmprlb(), errclb(), formk(), formt(), freev(), lnsrlb(), matupd(), prn1lb(), prn2lb(), prn3lb(), projgr(), subsm(), and timer().

Referenced by setulb().

Here is the call graph for this function:



Here is the caller graph for this function:

3.14 src/matupd.f File Reference

Functions/Subroutines

- subroutine matupd (n, m, ws, wy, sy, ss, d, r, itail, iupdat, col, head, theta, rr, dr, stp, dtd)

  This subroutine updates matrices WS and WY, and forms the middle matrix in B.

3.14.1 Function/Subroutine Documentation

3.14.1.1 matupd() subroutine matupd (

```
integer n,
integer m,
double precision, dimension(n, m) ws,
double precision, dimension(n, m) wy,
double precision, dimension(m, m) sy,
double precision, dimension(m, m) ss,
double precision, dimension(n) d,
double precision, dimension(n) r,
integer itail,
integer iupdat,
integer col,
integer head,
double precision theta,
double precision rr,
double precision dr,
double precision stp,
double precision dtd )
```

This subroutine updates matrices WS and WY, and forms the middle matrix in B.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit m is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.

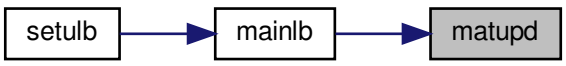
Parameters

wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
ss	On entry this stores S'S, that defines the limited memory BFGS matrix. On exit this array is unchanged.
d	TODO
r	TODO
itail	TODO
iupdat	TODO
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
rr	TODO
dr	TODO
stp	TODO
dtd	TODO

Definition at line 49 of file matupd.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.15 src/prn1lb.f File Reference

Functions/Subroutines

- subroutine prn1lb (n, m, l, u, x, iprint, itfile, epsmch)

  This subroutine prints the input data, initial point, upper and lower bounds of each variable, machine precision, as well as the headings of the output.

3.15.1 Function/Subroutine Documentation

3.15.1.1 prn1lb()

```
subroutine prn1lb (
    integer n,
    integer m,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    double precision, dimension(n) x,
    integer iprint,
    integer itfile,
    double precision epsmch )
```

This subroutine prints the input data, initial point, upper and lower bounds of each variable, machine precision, as well as the headings of the output.

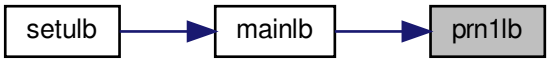
Parameters

n	On entry n is the number of variables. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit m is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
iprint	It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
itfile	unit number of iterate.dat file
epsmch	machine precision epsilon

Definition at line 39 of file prn1lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.16 src/prn2lb.f File Reference

Functions/Subroutines

- subroutine prn2lb (n, x, f, g, iprint, itfile, iter, nfgv, nact, sbgnrm, nseg, word, iword, iback, stp, xstep)

  This subroutine prints out new information after a successful line search.

3.16.1 Function/Subroutine Documentation

3.16.1.1 prn2lb()

```
subroutine prn2lb (
    integer n,
    double precision, dimension(n) x,
    double precision f,
    double precision, dimension(n) g,
    integer iprint,
    integer itfile,
    integer iter,
    integer nfgv,
    integer nact,
    double precision sbgnrm,
    integer nseg,
    character*3 word,
    integer iword,
    integer iback,
    double precision stp,
    double precision xstep )
```

This subroutine prints out new information after a successful line search.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
g	On first entry g is unspecified. On final exit g is the value of the gradient at x.
iprint	It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.

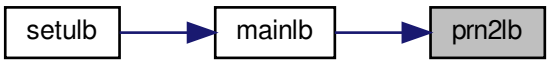
Parameters

itfile	unit number of iterate.dat file
iter	TODO
nfgv	TODO
nact	TODO
sbgnrm	TODO
nseg	TODO
word	TODO
iword	TODO
iback	TODO
stp	TODO
xstep	TODO

Definition at line 43 of file prn2lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.17 src/prn3lb.f File Reference

Functions/Subroutines

- subroutine prn3lb (n, x, f, task, iprint, info, itfile, iter, nfgv, nintol, nskip, nact, sbgnrm, time, nseg, word, iback, stp, xstep, k, cachyt, sbtime, lnscht)

  This subroutine prints out information when either a built-in convergence test is satisfied or when an error message is generated.

3.17.1 Function/Subroutine Documentation

3.17.1.1 prn3lb()

```
subroutine prn3lb (
    integer n,
    double precision, dimension(n) x,
    double precision f,
    character*60 task,
    integer iprint,
    integer info,
    integer itfile,
    integer iter,
    integer nfgv,
    integer nintol,
    integer nskip,
    integer nact,
    double precision sbgnrm,
    double precision time,
    integer nseg,
    character*3 word,
    integer iback,
    double precision stp,
    double precision xstep,
    integer k,
    double precision cachyt,
    double precision sbtime,
    double precision lnscht )
```

This subroutine prints out information when either a built-in convergence test is satisfied or when an error message is generated.

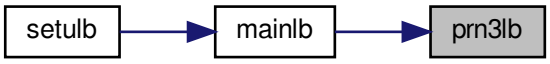
Parameters

n	On entry n is the number of variables. On exit n is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
task	working string indicating the current job when entering and leaving this subroutine.
iprint	TODO
info	TODO
itfile	unit number of iterate.dat file
iter	TODO
nfgv	TODO
nintol	TODO
nskip	TODO
nact	TODO
sbgnrm	TODO
time	TODO
nseg	TODO
word	TODO
iback	TODO
stp	TODO
xstep	TODO
k	TODO
cachyt	TODO
sbtime	TODO
lnscht	TODO

Definition at line 41 of file prn3lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.18 src/projgr.f File Reference

Functions/Subroutines

- subroutine projgr (n, l, u, nbd, x, g, sbgnrm)

  This subroutine computes the infinity norm of the projected gradient.

3.18.1 Function/Subroutine Documentation

3.18.1.1 projgr()

```
subroutine projgr (
    integer n,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    double precision, dimension(n) x,
    double precision, dimension(n) g,
    double precision sbgnrm )
```

This subroutine computes the infinity norm of the projected gradient.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, • 3 if x(i) has only an upper bound. On exit nbd is unchanged.

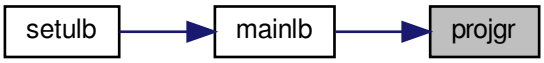
Parameters

x	On entry x is an approximation to the solution. On exit x is unchanged.
g	On entry g is the gradient. On exit g is unchanged.
sbgnrm	infinity norm of projected gradient

Definition at line 33 of file projgr.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.19 src/setulb.f File Reference

Functions/Subroutines

- subroutine setulb (n, m, x, l, u, nbd, f, g, factr, pgtol, wa, iwa, task, iprint, csave, lsave, isave, dsave)

  This subroutine partitions the working arrays wa and iwa, and then uses the limited memory BFGS method to solve the bound constrained optimization problem by calling mainlb.

3.19.1 Function/Subroutine Documentation

3.19.1.1 setulb()

```
subroutine setulb (
        integer n,
        integer m,
        double precision, dimension(n) x,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision f,
        double precision, dimension(n) g,
        double precision factr,
        double precision pgtol,
        double precision, dimension(2*m*n + 5*n + 11*m*m + 8*m) wa,
        integer, dimension(3*n) iwa,
        character*60 task,
        integer iprint,
```

```
character*60 csave,
logical, dimension(4) lsave,
integer, dimension(44) isave,
double precision, dimension(29) dsave )
```

This subroutine partitions the working arrays wa and iwa, and then uses the limited memory BFGS method to solve the bound constrained optimization problem by calling mainlb. (The direct method will be used in the subspace minimization.)

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
l	On entry l is the lower bound on x. On exit l is unchanged.
u	On entry u is the upper bound on x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
g	On first entry g is unspecified. On final exit g is the value of the gradient at x.
factr	On entry factr >= 0 is specified by the user. The iteration will stop when (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch where epsmch is the machine precision, which is automatically generated by the code. Typical values for factr: • 1.d+12 for low accuracy; • 1.d+7 for moderate accuracy; • 1.d+1 for extremely high accuracy. On exit factr is unchanged.
pgtol	On entry pgtol >= 0 is specified by the user. The iteration will stop when max{|proj g_i | i = 1, ..., n} <= pgtol where pg_i is the ith component of the projected gradient. On exit pgtol is unchanged.
wa	working array
iwa	working array
task	working string indicating the current job when entering and quitting this subroutine.

Parameters

iprint	Must be set by the user. It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
csave	working string
lsave	working array; On exit with 'task' = NEW_X, the following information is available: • If lsave(1) = .true. then the initial X has been replaced by its projection in the feasible set; • If lsave(2) = .true. then the problem is constrained; • If lsave(3) = .true. then each variable has upper and lower bounds;
isave	working array; On exit with 'task' = NEW_X, the following information is available: • isave(22) = the total number of intervals explored in the search of Cauchy points; • isave(26) = the total number of skipped BFGS updates before the current iteration; • isave(30) = the number of current iteration; • isave(31) = the total number of BFGS updates prior the current iteration; • isave(33) = the number of intervals explored in the search of Cauchy point in the current iteration; • isave(34) = the total number of function and gradient evaluations; • isave(36) = the number of function value or gradient evaluations in the current iteration; • if isave(37) = 0 then the subspace argmin is within the box; • if isave(37) = 1 then the subspace argmin is beyond the box; • isave(38) = the number of free variables in the current iteration; • isave(39) = the number of active constraints in the current iteration; • n + 1 - isave(40) = the number of variables leaving the set of active constraints in the current iteration; • isave(41) = the number of variables entering the set of active constraints in the current iteration.

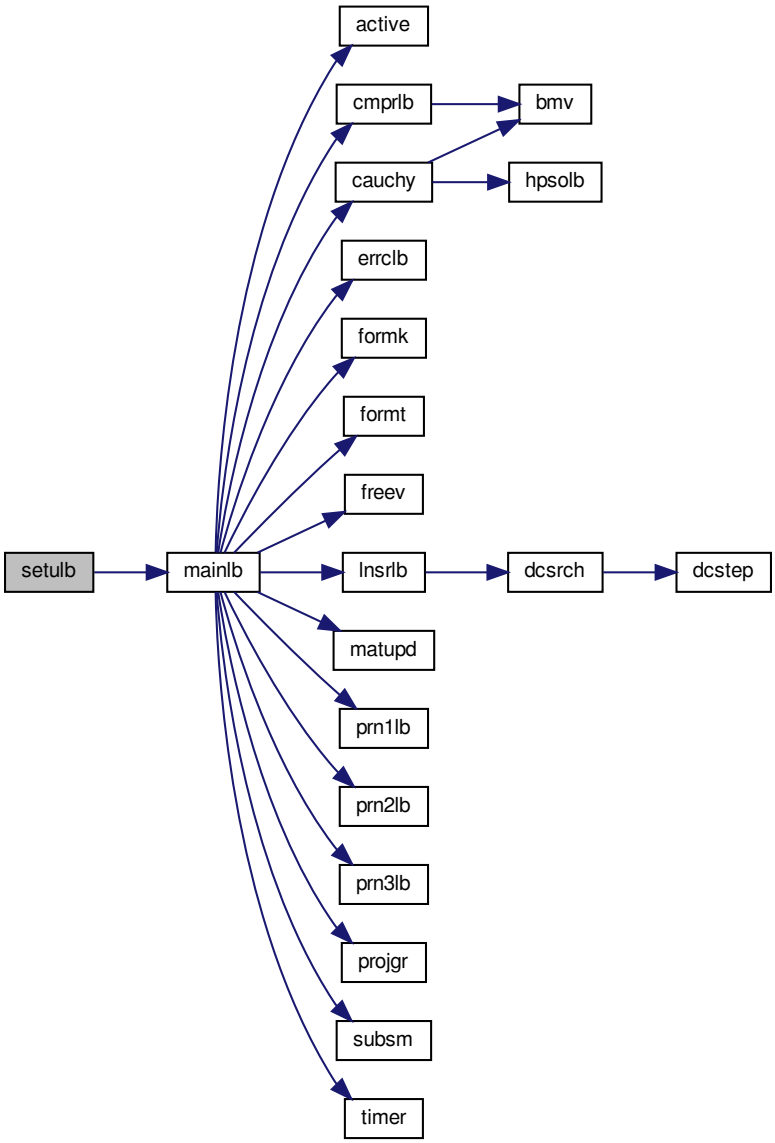
Parameters

dsave	working array; On exit with 'task' = NEW_X, the following information is available: • dsave(1) = current 'theta' in the BFGS matrix; • dsave(2) = f(x) in the previous iteration; • dsave(3) = factr∗epsmch; • dsave(4) = 2-norm of the line search direction vector; • dsave(5) = the machine precision epsmch generated by the code; • dsave(7) = the accumulated time spent on searching for Cauchy points; • dsave(8) = the accumulated time spent on subspace minimization; • dsave(9) = the accumulated time spent on line search; • dsave(11) = the slope of the line search function at the current point of line search; • dsave(12) = the maximum relative step length imposed in line search; • dsave(13) = the infinity norm of the projected gradient; • dsave(14) = the relative step length in the line search; • dsave(15) = the slope of the line search function at the starting point of the line search; • dsave(16) = the square of the 2-norm of the line search direction vector.

Definition at line 188 of file setulb.f.

References mainlb().

Here is the call graph for this function:



3.20 src/subsm.f File Reference

Functions/Subroutines

- subroutine subsm (n, m, nsub, ind, l, u, nbd, x, d, xp, ws, wy, theta, xx, gg, col, head, iword, wv, wn, iprint, info)

  Performs the subspace minimization.

3.20.1 Function/Subroutine Documentation

3.20.1.1 subsm()

```
subroutine subsm (
        integer n,
        integer m,
        integer nsub,
        integer, dimension(nsub) ind,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision, dimension(n) x,
        double precision, dimension(n) d,
        double precision, dimension(n) xp,
        double precision, dimension(n, m) ws,
        double precision, dimension(n, m) wy,
        double precision theta,
        double precision, dimension(n) xx,
        double precision, dimension(n) gg,
        integer col,
        integer head,
        integer iword,
        double precision, dimension(2*m) wv,
        double precision, dimension(2*m, 2*m) wn,
        integer iprint,
        integer info )
```

Given xcp, l, u, r, an index set that specifies the active set at xcp, and an l-BFGS matrix B (in terms of WY, WS, SY, WT, head, col, and theta), this subroutine computes an approximate solution of the subspace problem

(P) min Q(x) = r'(x-xcp) + 1/2 (x-xcp)' B (x-xcp)

subject to l<=x<=u x_i=xcp_i for all i in A(xcp)

along the subspace unconstrained Newton direction

d = -(Z'BZ)^(-1) r.

The formula for the Newton direction, given the L-BFGS matrix and the Sherman-Morrison formula, is

d = (1/theta)r + (1/theta*2) Z'WK^(-1)W'Z r.

where K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta*S'AA'S]

Note that this procedure for computing d differs from that described in [1]. One can show that the matrix K is equal to the matrix M^[-1]N in that paper.

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
nsub	On entry nsub is the number of free variables. On exit nsub is unchanged.
ind	On entry ind specifies the coordinate indices of free variables. On exit ind is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.

Parameters

u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
x	On entry x specifies the Cauchy point xcp. On exit x(i) is the minimizer of Q over the subspace of free variables.
d	On entry d is the reduced gradient of Q at xcp. On exit d is the Newton direction of Q.
xp	used to safeguard the projected Newton direction
xx	On entry it holds the current iterate. On output it is unchanged.
gg	On entry it holds the gradient at the current iterate. On output it is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
iword	On entry iword is unspecified. On exit iword specifies the status of the subspace solution. iword = • 0 if the solution is in the box, • 1 if some bound is encountered.
wv	working array
wn	On entry the upper triangle of wn stores the LEL^T factorization of the indefinite matrix K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta∗S'AA'S] where E = [-I 0] [0 I] On exit wn is unchanged.

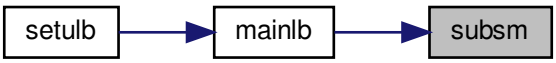
Parameters

iprint	must be set by the user; It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
info	On entry info is unspecified. On exit info = • 0 for normal return, • nonzero for abnormal return when the matrix K is ill-conditioned.

Definition at line 124 of file subsm.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.21 src/timer.f File Reference

Functions/Subroutines

- subroutine timer (ttime)

  This routine computes cpu time in double precision.

3.21.1 Function/Subroutine Documentation

3.21.1.1 timer()

```
subroutine timer (
        double precision ttime )
```

This routine computes cpu time in double precision; it makes use of the intrinsic f90 cpu_time therefore a conversion type is needed.

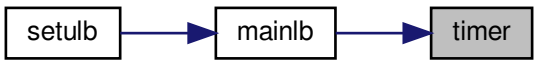
Parameters

ttime	CPU time in double precision

Definition at line 10 of file timer.f.

Referenced by mainlb().

Here is the caller graph for this function:



4 Example Documentation

4.1 driver1.f

This simple driver demonstrates how to call the L-BFGS-B code to solve a sample problem (the extended Rosenbrock function subject to bounds on the variables). The dimension n of this problem is variable. (Fortran-77 version)

```
1 c> \file driver1.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                           DRIVER 1 in Fortran 77
9 c     ----------------------------------------------------------------
10 c               SIMPLE DRIVER FOR L-BFGS-B (version 3.0)
11 c     ----------------------------------------------------------------
12 c
13 c        L-BFGS-B is a code for solving large nonlinear optimization
14 c             problems with simple bounds on the variables.
15 c
16 c        The code can also be used for unconstrained problems and is
17 c        as efficient for these problems as the earlier limited memory
18 c                         code L-BFGS.
19 c
20 c        This is the simplest driver in the package. It uses all the
21 c                    default settings of the code.
22 c
23 c
24 c     References:
25 c
26 c        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 c        memory algorithm for bound constrained optimization",
28 c        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 c
30 c        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 c        Subroutines for Large Scale Bound Constrained Optimization"
32 c        Tech. Report, NAM-11, EECS Department, Northwestern University,
33 c        1994.
34 c
35 c
36 c          (Postscript files of these papers are available via anonymous
37 c           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 c
39 c                               *  *  *
40 c
41 c         March 2011   (latest revision)
```

```
c          Optimization Center at Northwestern University
c          Instituto Tecnologico Autonomo de Mexico
c
c          Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
c          L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
c          Optimization"  (2011). To appear in  ACM Transactions on
c          Mathematical Software,
c
c     ============================================================
c             DESCRIPTION OF THE VARIABLES IN L-BFGS-B
c     ============================================================
c
c     n is an INTEGER variable that must be set by the user to the
c       number of variables.  It is not altered by the routine.
c
c     m is an INTEGER variable that must be set by the user to the
c       number of corrections used in the limited memory matrix.
c       It is not altered by the routine.  Values of m < 3  are
c       not recommended, and large values of m can result in excessive
c       computing time. The range  3 <= m <= 20 is recommended.
c
c     x is a DOUBLE PRECISION array of length n.  On initial entry
c       it must be set by the user to the values of the initial
c       estimate of the solution vector.  Upon successful exit, it
c       contains the values of the variables at the best point
c       found (usually an approximate solution).
c
c     l is a DOUBLE PRECISION array of length n that must be set by
c       the user to the values of the lower bounds on the variables. If
c       the i-th variable has no lower bound, l(i) need not be defined.
c
c     u is a DOUBLE PRECISION array of length n that must be set by
c       the user to the values of the upper bounds on the variables. If
c       the i-th variable has no upper bound, u(i) need not be defined.
c
c     nbd is an INTEGER array of dimension n that must be set by the
c       user to the type of bounds imposed on the variables:
c       nbd(i)=0 if x(i) is unbounded,
c              1 if x(i) has only a lower bound,
c              2 if x(i) has both lower and upper bounds,
c              3 if x(i) has only an upper bound.
c
c     f is a DOUBLE PRECISION variable.  If the routine setulb returns
c       with task(1:2)= 'FG', then f must be set by the user to
c       contain the value of the function at the point x.
c
c     g is a DOUBLE PRECISION array of length n.  If the routine setulb
c       returns with taskb(1:2)= 'FG', then g must be set by the user to
c       contain the components of the gradient at the point x.
c
c     factr is a DOUBLE PRECISION variable that must be set by the user.
c       It is a tolerance in the termination test for the algorithm.
c       The iteration will stop when
c
c         (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch
c
c       where epsmch is the machine precision which is automatically
c       generated by the code. Typical values for factr on a computer
c        with 15 digits of accuracy in double precision are:
c        factr=1.d+12 for low accuracy;
c              1.d+7  for moderate accuracy;
c              1.d+1  for extremely high accuracy.
c       The user can suppress this termination test by setting factr=0.
c
c     pgtol is a double precision variable.
c       On entry pgtol >= 0 is specified by the user.  The iteration
c         will stop when
c
c                 max{|proj g_i | i = 1, ..., n} <= pgtol
c
c         where pg_i is the ith component of the projected gradient.
c       The user can suppress this termination test by setting pgtol=0.
c
c     wa is a DOUBLE PRECISION  array of length
c       (2mmax + 5)nmax + 11mmax^2 + 8mmax used as workspace.
c       This array must not be altered by the user.
c
c     iwa is an INTEGER  array of length 3nmax used as
c       workspace. This array must not be altered by the user.
c
c     task is a CHARACTER string of length 60.
c       On first entry, it must be set to 'START'.
c       On a return with task(1:2)='FG', the user must evaluate the
c         function f and gradient g at the returned value of x.
c       On a return with task(1:5)='NEW_X', an iteration of the
c         algorithm has concluded, and f and g contain f(x) and g(x)
c         respectively.  The user can decide whether to continue or stop
```

```
129 c         the iteration.
130 c       When
131 c         task(1:4)='CONV', the termination test in L-BFGS-B has been
132 c           satisfied;
133 c         task(1:4)='ABNO', the routine has terminated abnormally
134 c           without being able to satisfy the termination conditions,
135 c           x contains the best approximation found,
136 c           f and g contain f(x) and g(x) respectively;
137 c         task(1:5)='ERROR', the routine has detected an error in the
138 c           input parameters;
139 c       On exit with task = 'CONV', 'ABNO' or 'ERROR', the variable task
140 c         contains additional information that the user can print.
141 c       This array should not be altered unless the user wants to
142 c          stop the run for some reason.  See driver2 or driver3
143 c          for a detailed explanation on how to stop the run
144 c          by assigning task(1:4)='STOP' in the driver.
145 c
146 c     iprint is an INTEGER variable that must be set by the user.
147 c       It controls the frequency and type of output generated:
148 c        iprint<0    no output is generated;
149 c        iprint=0    print only one line at the last iteration;
150 c        0<iprint<99 print also f and |proj g| every iprint iterations;
151 c        iprint=99   print details of every iteration except n-vectors;
152 c        iprint=100  print also the changes of active set and final x;
153 c        iprint>100  print details of every iteration including x and g;
154 c       When iprint > 0, the file iterate.dat will be created to
155 c                        summarize the iteration.
156 c
157 c     csave  is a CHARACTER working array of length 60.
158 c
159 c     lsave is a LOGICAL working array of dimension 4.
160 c       On exit with task = 'NEW_X', the following information is
161 c         available:
162 c       lsave(1) = .true.  the initial x did not satisfy the bounds;
163 c       lsave(2) = .true.  the problem contains bounds;
164 c       lsave(3) = .true.  each variable has upper and lower bounds.
165 c
166 c     isave is an INTEGER working array of dimension 44.
167 c       On exit with task = 'NEW_X', it contains information that
168 c       the user may want to access:
169 c         isave(30) = the current iteration number;
170 c         isave(34) = the total number of function and gradient
171 c                         evaluations;
172 c         isave(36) = the number of function value or gradient
173 c                                  evaluations in the current iteration;
174 c         isave(38) = the number of free variables in the current
175 c                         iteration;
176 c         isave(39) = the number of active constraints at the current
177 c                         iteration;
178 c
179 c         see the subroutine setulb.f for a description of other
180 c         information contained in isave
181 c
182 c     dsave is a DOUBLE PRECISION working array of dimension 29.
183 c       On exit with task = 'NEW_X', it contains information that
184 c         the user may want to access:
185 c         dsave(2) = the value of f at the previous iteration;
186 c         dsave(5) = the machine precision epsmch generated by the code;
187 c         dsave(13) = the infinity norm of the projected gradient;
188 c
189 c         see the subroutine setulb.f for a description of other
190 c         information contained in dsave
191 c
192 c     ------------------------------------------------------------
193 c           END OF THE DESCRIPTION OF THE VARIABLES IN L-BFGS-B
194 c     ------------------------------------------------------------
195
196       program driver
197       use json_writer
198
199 c     This simple driver demonstrates how to call the L-BFGS-B code to
200 c       solve a sample problem (the extended Rosenbrock function
201 c       subject to bounds on the variables). The dimension n of this
202 c       problem is variable.
203
204       integer          nmax, mmax
205       parameter(nmax=1024, mmax=17)
206 c        nmax is the dimension of the largest problem to be solved.
207 c        mmax is the maximum number of limited memory corrections.
208
209 c     Declare the variables needed by the code.
210 c       A description of all these variables is given at the end of
211 c       the driver.
212
213       character*60     task, csave
214       logical          lsave(4)
215       integer          n, m, iprint,
```

```
216      +                 nbd(nmax), iwa(3*nmax), isave(44)
217       double precision f, factr, pgtol,
218      +                 x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
219      +                 wa(2*mmax*nmax + 5*nmax + 11*mmax*mmax + 8*mmax)
220
221 c     Declare a few additional variables for this sample problem.
222
223       double precision t1, t2
224       integer          i
225
226 c     Test instrumentation: dump per-iteration state to JSON when the
227 c     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
228       character*512    lbfgsb_json
229       logical          json_active
230
231 c     We wish to have output at every iteration.
232
233       iprint = 1
234
235 c     We specify the tolerances in the stopping criteria.
236
237       factr=1.0d+7
238       pgtol=1.0d-5
239
240 c     We specify the dimension n of the sample problem and the number
241 c        m of limited memory corrections stored.  (n and m should not
242 c        exceed the limits nmax and mmax respectively.)
243
244       n=25
245       m=5
246
247 c     We now provide nbd which defines the bounds on the variables:
248 c                     l   specifies the lower bounds,
249 c                     u   specifies the upper bounds.
250
251 c     First set bounds on the odd-numbered variables.
252
253       do 10 i=1,n,2
254          nbd(i)=2
255          l(i)=1.0d0
256          u(i)=1.0d2
257    10 continue
258
259 c     Next set bounds on the even-numbered variables.
260
261       do 12 i=2,n,2
262          nbd(i)=2
263          l(i)=-1.0d2
264          u(i)=1.0d2
265    12  continue
266
267 c     We now define the starting point.
268
269       do 14 i=1,n
270          x(i)=3.0d0
271    14 continue
272
273       write (6,16)
274    16 format(/,5x, 'Solving sample problem.',
275      +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
276
277 c     We start the iteration by initializing task.
278 c
279       task = 'START'
280
281 c     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
282       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
283       json_active = (len_trim(lbfgsb_json) .gt. 0)
284
285 c        ------- the beginning of the loop ----------
286
287  111  continue
288
289 c     This is the call to the L-BFGS-B code.
290
291       call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
292      +            csave,lsave,isave,dsave)
293
294       if (task(1:2) .eq. 'FG') then
295 c        the minimization routine has returned to request the
296 c        function f and gradient g values at the current x.
297
298 c        Compute function value f for the sample problem.
299
300          f=.25d0*(x(1)-1.d0)**2
301          do 20 i=2,n
302             f=f+(x(i)-x(i-1)**2)**2
```

```
303   20     continue
304          f=4.d0*f
305
306 c        Compute gradient g for the sample problem.
307
308          t1=x(2)-x(1)**2
309          g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
310          do 22 i=2,n-1
311             t2=t1
312             t1=x(i+1)-x(i)**2
313             g(i)=8.d0*t2-1.6d1*x(i)*t1
314   22     continue
315          g(n)=8.d0*t1
316
317 c          go back to the minimization routine.
318          goto 111
319       endif
320 c
321       if (task(1:5) .eq. 'NEW_X')  goto 111
322 c        the minimization routine has returned with a new iterate,
323 c         and we have opted to continue the iteration.
324
325 c           ---------- the end of the loop -------------
326
327 c     If task is neither FG nor NEW_X we terminate execution.
328
329       if (json_active) then
330          call json_write_aggregate(trim(lbfgsb_json), task, f,
331      +                             dsave(13))
332       endif
333
334       stop
335
336       end
337
338 c======================= The end of driver1 ============================
```

4.2 driver1.f90

This simple driver demonstrates how to call the L-BFGS-B code to solve a sample problem (the extended Rosenbrock function subject to bounds on the variables). The dimension n of this problem is variable. (Fortran-90 version)

```
1 !> \file driver1.f90
2
3 !
4 !  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 !  or "3-clause license")
6 !  Please read attached file License.txt
7 !
8 !
9 !                       DRIVER1 in Fortran 90
10 !     ----------------------------------------------------------------
11 !
12 !        L-BFGS-B is a code for solving large nonlinear optimization
13 !             problems with simple bounds on the variables.
14 !
15 !        The code can also be used for unconstrained problems and is
16 !        as efficient for these problems as the earlier limited memory
17 !                         code L-BFGS.
18 !
19 !        This is the simplest driver in the package. It uses all the
20 !                    default settings of the code.
21 !
22 !
23 !     References:
24 !
25 !        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
26 !        memory algorithm for bound constrained optimization",
27 !        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
28 !
29 !        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
30 !        Subroutines for Large Scale Bound Constrained Optimization"
31 !        Tech. Report, NAM-11, EECS Department, Northwestern University,
32 !        1994.
33 !
34 !
35 !          (Postscript files of these papers are available via anonymous
36 !           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
37 !
38 !                               *  *  *
39 !
40 !         March 2011   (latest revision)
41 !         Optimization Center at Northwestern University
```

```
!          Instituto Tecnologico Autonomo de Mexico
!
!          Jorge Nocedal and Jose Luis Morales
!
!     ------------------------------------------------------------
!             DESCRIPTION OF THE VARIABLES IN L-BFGS-B
!     ------------------------------------------------------------
!
!     n is an INTEGER variable that must be set by the user to the
!       number of variables.  It is not altered by the routine.
!
!     m is an INTEGER variable that must be set by the user to the
!       number of corrections used in the limited memory matrix.
!       It is not altered by the routine.  Values of m < 3  are
!       not recommended, and large values of m can result in excessive
!       computing time. The range  3 <= m <= 20 is recommended.
!
!     x is a DOUBLE PRECISION array of length n.  On initial entry
!       it must be set by the user to the values of the initial
!       estimate of the solution vector.  Upon successful exit, it
!       contains the values of the variables at the best point
!       found (usually an approximate solution).
!
!     l is a DOUBLE PRECISION array of length n that must be set by
!       the user to the values of the lower bounds on the variables. If
!       the i-th variable has no lower bound, l(i) need not be defined.
!
!     u is a DOUBLE PRECISION array of length n that must be set by
!       the user to the values of the upper bounds on the variables. If
!       the i-th variable has no upper bound, u(i) need not be defined.
!
!     nbd is an INTEGER array of dimension n that must be set by the
!       user to the type of bounds imposed on the variables:
!       nbd(i)=0 if x(i) is unbounded,
!              1 if x(i) has only a lower bound,
!              2 if x(i) has both lower and upper bounds,
!              3 if x(i) has only an upper bound.
!
!     f is a DOUBLE PRECISION variable.  If the routine setulb returns
!       with task(1:2)= 'FG', then f must be set by the user to
!       contain the value of the function at the point x.
!
!     g is a DOUBLE PRECISION array of length n.  If the routine setulb
!       returns with taskb(1:2)= 'FG', then g must be set by the user to
!       contain the components of the gradient at the point x.
!
!     factr is a DOUBLE PRECISION variable that must be set by the user.
!       It is a tolerance in the termination test for the algorithm.
!       The iteration will stop when
!
!        (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch
!
!       where epsmch is the machine precision which is automatically
!       generated by the code. Typical values for factr on a computer
!       with 15 digits of accuracy in double precision are:
!       factr=1.d+12 for low accuracy;
!             1.d+7  for moderate accuracy;
!             1.d+1  for extremely high accuracy.
!      The user can suppress this termination test by setting factr=0.
!
!     pgtol is a double precision variable.
!       On entry pgtol >= 0 is specified by the user.  The iteration
!         will stop when
!
!                 max{|proj g_i | i = 1, ..., n} <= pgtol
!
!         where pg_i is the ith component of the projected gradient.
!       The user can suppress this termination test by setting pgtol=0.
!
!     wa is a DOUBLE PRECISION  array of length
!       (2mmax + 5)nmax + 11mmax^2 + 8mmax used as workspace.
!       This array must not be altered by the user.
!
!     iwa is an INTEGER  array of length 3nmax used as
!       workspace. This array must not be altered by the user.
!
!     task is a CHARACTER string of length 60.
!       On first entry, it must be set to 'START'.
!       On a return with task(1:2)='FG', the user must evaluate the
!         function f and gradient g at the returned value of x.
!       On a return with task(1:5)='NEW_X', an iteration of the
!         algorithm has concluded, and f and g contain f(x) and g(x)
!         respectively.  The user can decide whether to continue or stop
!         the iteration.
!       When
!         task(1:4)='CONV', the termination test in L-BFGS-B has been
!           satisfied;
```

```
129 !         task(1:4)='ABNO', the routine has terminated abnormally
130 !           without being able to satisfy the termination conditions,
131 !           x contains the best approximation found,
132 !           f and g contain f(x) and g(x) respectively;
133 !         task(1:5)='ERROR', the routine has detected an error in the
134 !           input parameters;
135 !       On exit with task = 'CONV', 'ABNO' or 'ERROR', the variable task
136 !         contains additional information that the user can print.
137 !       This array should not be altered unless the user wants to
138 !          stop the run for some reason.  See driver2 or driver3
139 !          for a detailed explanation on how to stop the run
140 !          by assigning task(1:4)='STOP' in the driver.
141 !
142 !     iprint is an INTEGER variable that must be set by the user.
143 !       It controls the frequency and type of output generated:
144 !        iprint<0    no output is generated;
145 !        iprint=0    print only one line at the last iteration;
146 !        0<iprint<99 print also f and |proj g| every iprint iterations;
147 !        iprint=99   print details of every iteration except n-vectors;
148 !        iprint=100  print also the changes of active set and final x;
149 !        iprint>100  print details of every iteration including x and g;
150 !       When iprint > 0, the file iterate.dat will be created to
151 !                        summarize the iteration.
152 !
153 !     csave  is a CHARACTER working array of length 60.
154 !
155 !     lsave is a LOGICAL working array of dimension 4.
156 !       On exit with task = 'NEW_X', the following information is
157 !         available:
158 !       lsave(1) = .true.  the initial x did not satisfy the bounds;
159 !       lsave(2) = .true.  the problem contains bounds;
160 !       lsave(3) = .true.  each variable has upper and lower bounds.
161 !
162 !     isave is an INTEGER working array of dimension 44.
163 !       On exit with task = 'NEW_X', it contains information that
164 !       the user may want to access:
165 !         isave(30) = the current iteration number;
166 !         isave(34) = the total number of function and gradient
167 !                          evaluations;
168 !         isave(36) = the number of function value or gradient
169 !                                  evaluations in the current iteration;
170 !         isave(38) = the number of free variables in the current
171 !                          iteration;
172 !         isave(39) = the number of active constraints at the current
173 !                          iteration;
174 !
175 !         see the subroutine setulb.f for a description of other
176 !         information contained in isave
177 !
178 !     dsave is a DOUBLE PRECISION working array of dimension 29.
179 !       On exit with task = 'NEW_X', it contains information that
180 !         the user may want to access:
181 !         dsave(2)  = the value of f at the previous iteration;
182 !         dsave(5)  = the machine precision epsmch generated by the code;
183 !         dsave(13) = the infinity norm of the projected gradient;
184 !
185 !         see the subroutine setulb.f for a description of other
186 !         information contained in dsave
187 !
188 !     ------------------------------------------------------------
189 !           END OF THE DESCRIPTION OF THE VARIABLES IN L-BFGS-B
190 !     ------------------------------------------------------------
191 !
192       program driver
193       use json_writer
194 !
195 !     This simple driver demonstrates how to call the L-BFGS-B code to
196 !       solve a sample problem (the extended Rosenbrock function
197 !       subject to bounds on the variables). The dimension n of this
198 !       problem is variable.
199
200       implicit none
201 !
202 !     Declare variables and parameters needed by the code.
203 !       Note thar we wish to have output at every iteration.
204 !          iprint=1
205 !
206 !       We also specify the tolerances in the stopping criteria.
207 !          factr  = 1.0d+7, pgtol  = 1.0d-5
208 !
209 !       A description of all these variables is given at the beginning
210 !       of  the driver
211 !
212       integer,  parameter    :: n = 25, m = 5, iprint = 1
213       integer,  parameter    :: dp = kind(1.0d0)
214       real(dp), parameter    :: factr  = 1.0d+7, pgtol  = 1.0d-5
215 !
```

```
216       character(len=60)      :: task, csave
217       logical                :: lsave(4)
218       integer                :: isave(44)
219       real(dp)               :: f
220       real(dp)               :: dsave(29)
221       integer,  allocatable  :: nbd(:), iwa(:)
222       real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)
223
224 !     Declare a few additional variables for this sample problem
225
226       real(dp)               :: t1, t2
227       integer                :: i
228
229 !     Test instrumentation: dump per-iteration state to JSON when the
230 !     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
231       character(len=512)     :: lbfgsb_json
232       logical                :: json_active
233
234 !     Allocate dynamic arrays
235
236       allocate ( nbd(n), x(n), l(n), u(n), g(n) )
237       allocate ( iwa(3*n) )
238       allocate ( wa(2*m*n + 11*m*m + 5*n + 8*m) )
239 !
240       do 10 i=1, n, 2
241          nbd(i) = 2
242          l(i)   = 1.0d0
243          u(i)   = 1.0d2
244    10 continue
245
246 !     Next set bounds on the even-numbered variables.
247
248       do 12 i=2, n, 2
249          nbd(i) =  2
250          l(i)   = -1.0d2
251          u(i)   =  1.0d2
252    12  continue
253
254 !     We now define the starting point.
255
256       do 14 i=1, n
257          x(i) = 3.0d0
258    14 continue
259
260       write (6,16)
261    16 format(/,5x, 'Solving sample problem.', &
262                /,5x, ' (f = 0.0 at the optimal solution.)',/)
263
264 !     We start the iteration by initializing task.
265
266       task = 'START'
267
268 !     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
269       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
270       json_active = (len_trim(lbfgsb_json) > 0)
271
272 !     The beginning of the loop
273
274       do while(task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
275                task.eq.'START')
276
277 !     This is the call to the L-BFGS-B code.
278
279          call setulb ( n, m, x, l, u, nbd, f, g, factr, pgtol, &
280                        wa, iwa, task, iprint,&
281                        csave, lsave, isave, dsave )
282
283          if (task(1:2) .eq. 'FG') then
284
285             f=.25d0*( x(1)-1.d0 )**2
286             do 20 i=2, n
287                f = f + ( x(i)-x(i-1 )**2 )**2
288    20       continue
289             f = 4.d0*f
290
291 !     Compute gradient g for the sample problem.
292
293             t1   = x(2) - x(1)**2
294             g(1) = 2.d0*(x(1) - 1.d0) - 1.6d1*x(1)*t1
295             do 22 i=2, n-1
296                t2   = t1
297                t1   = x(i+1) - x(i)**2
298                g(i) = 8.d0*t2 - 1.6d1*x(i)*t1
299    22       continue
300             g(n) = 8.d0*t1
301
302          end if
```

```
303       end do
304
305 !     end of loop do while
306
307       if (json_active) &
308          call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13))
309
310       end program driver
311
```

4.3 driver2.f

This driver shows how to replace the default stopping test by other termination criteria. It also illustrates how to print the values of several parameters during the course of the iteration. The sample problem used here is the same as in DRIVER1 (the extended Rosenbrock function with bounds on the variables). (Fortran-77 version)

```
1 c> \file driver2.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                           DRIVER 2 in Fortran 77
9 c     ----------------------------------------------------------------
10 c             CUSTOMIZED DRIVER FOR L-BFGS-B (version 3.0)
11 c     ----------------------------------------------------------------
12 c
13 c        L-BFGS-B is a code for solving large nonlinear optimization
14 c             problems with simple bounds on the variables.
15 c
16 c        The code can also be used for unconstrained problems and is
17 c        as efficient for these problems as the earlier limited memory
18 c                          code L-BFGS.
19 c
20 c        This driver illustrates how to control the termination of the
21 c        run and how to design customized output.
22 c
23 c     References:
24 c
25 c        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
26 c        memory algorithm for bound constrained optimization",
27 c        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
28 c
29 c        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
30 c        Subroutines for Large Scale Bound Constrained Optimization"
31 c        Tech. Report, NAM-11, EECS Department, Northwestern University,
32 c        1994.
33 c
34 c
35 c          (Postscript files of these papers are available via anonymous
36 c           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
37 c
38 c                                * *  *
39 c
40 c         February 2011   (latest revision)
41 c         Optimization Center at Northwestern University
42 c         Instituto Tecnologico Autonomo de Mexico
43 c
44 c         Jorge Nocedal and Jose Luis Morales
45 c         Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c         L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c         Optimization"  (2011). To appear in  ACM Transactions on
48 c         Mathematical Software,
49 c
50 c     **************
51
52       program driver
53       use json_writer
54
55 c     This driver shows how to replace the default stopping test
56 c       by other termination criteria. It also illustrates how to
57 c       print the values of several parameters during the course of
58 c       the iteration. The sample problem used here is the same as in
59 c       DRIVER1 (the extended Rosenbrock function with bounds on the
60 c       variables).
61
62       integer          nmax, mmax
63       parameter(nmax=1024, mmax=17)
64 c        nmax is the dimension of the largest problem to be solved.
65 c        mmax is the maximum number of limited memory corrections.
66
```

```
67 c     Declare the variables needed by the code.
68 c       A description of all these variables is given at the end of
69 c       driver1.
70
71       character*60     task, csave
72       logical          lsave(4)
73       integer          n, m, iprint,
74      +                 nbd(nmax), iwa(3*nmax), isave(44)
75       double precision f, factr, pgtol,
76      +                 x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
77      +                 wa(2*mmax*nmax+5*nmax+11*mmax*mmax+8*mmax)
78
79 c     Declare a few additional variables for the sample problem.
80
81       double precision t1, t2
82       integer          i
83
84 c     Test instrumentation: dump aggregate end-of-run state to JSON when
85 c     the environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
86 c     LBFGSB_NFG_LIMIT optionally overrides the nfg stopping threshold.
87       character*512    lbfgsb_json
88       character*64     env_nfg_limit
89       logical          json_active
90       integer          nfg_limit
91
92 c     We suppress the default output.
93
94       iprint = -1
95
96 c     We suppress both code-supplied stopping tests because the
97 c        user is providing his own stopping criteria.
98
99       factr=0.0d0
100       pgtol=0.0d0
101
102 c     We specify the dimension n of the sample problem and the number
103 c        m of limited memory corrections stored.  (n and m should not
104 c        exceed the limits nmax and mmax respectively.)
105
106       n=25
107       m=5
108
109 c     We now specify nbd which defines the bounds on the variables:
110 c                     l   specifies the lower bounds,
111 c                     u   specifies the upper bounds.
112
113 c     First set bounds on the odd numbered variables.
114
115       do 10 i=1,n,2
116          nbd(i)=2
117          l(i)=1.0d0
118          u(i)=1.0d2
119   10  continue
120
121 c     Next set bounds on the even numbered variables.
122
123       do 12 i=2,n,2
124          nbd(i)=2
125          l(i)=-1.0d2
126          u(i)=1.0d2
127   12   continue
128
129 c     We now define the starting point.
130
131       do 14 i=1,n
132          x(i)=3.0d0
133   14  continue
134
135 c     We now write the heading of the output.
136
137       write (6,16)
138   16  format(/,5x, 'Solving sample problem.',
139      +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
140
141 c     We start the iteration by initializing task.
142 c
143       task = 'START'
144
145 c     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
146       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
147       json_active = (len_trim(lbfgsb_json) .gt. 0)
148       nfg_limit = 99
149       call get_environment_variable('LBFGSB_NFG_LIMIT', env_nfg_limit)
150       if (len_trim(env_nfg_limit) .gt. 0) then
151          read(env_nfg_limit, *) nfg_limit
152       endif
153
```

```
c        ------- the beginning of the loop ----------
 111  continue
c     This is the call to the L-BFGS-B code.
      call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
     +            csave,lsave,isave,dsave)
      if (task(1:2) .eq. 'FG') then
c        the minimization routine has returned to request the
c        function f and gradient g values at the current x.
c        Compute function value f for the sample problem.
         f=.25d0*(x(1)-1.d0)**2
         do 20 i=2,n
            f=f+(x(i)-x(i-1)**2)**2
  20     continue
         f=4.d0*f
c        Compute gradient g for the sample problem.
         t1=x(2)-x(1)**2
         g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
         do 22 i=2,n-1
            t2=t1
            t1=x(i+1)-x(i)**2
            g(i)=8.d0*t2-1.6d1*x(i)*t1
  22     continue
         g(n)=8.d0*t1
c          go back to the minimization routine.
         goto 111
      endif
c
      if (task(1:5) .eq. 'NEW_X') then
c
c        the minimization routine has returned with a new iterate.
c        At this point have the opportunity of stopping the iteration
c        or observing the values of certain parameters
c
c        First are two examples of stopping tests.
c        Note: task(1:4) must be assigned the value 'STOP' to terminate
c          the iteration and ensure that the final results are
c          printed in the default format. The rest of the character
c          string TASK may be used to store other information.
c        1) Terminate if the total number of f and g evaluations
c             exceeds 99.
         if (isave(34) .ge. nfg_limit)
     +      task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
c        2) Terminate if  |proj g|/(1+|f|) < 1.0d-10, where
c           "proj g" denoted the projected gradient
         if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f)))
     +      task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
c        We now wish to print the following information at each
c        iteration:
c          1) the current iteration number, isave(30),
c          2) the total number of f and g evaluations, isave(34),
c          3) the value of the objective function f,
c          4) the norm of the projected gradient,  dsve(13)
c        See the comments at the end of driver1 for a description
c        of the variables isave and dsave.
         write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate'
     +      ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
c        If the run is to be terminated, we print also the information
c        contained in task as well as the final value of x.
         if (task(1:4) .eq. 'STOP') then
            write (6,*) task
            write (6,*) 'Final X='
            write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
         endif
c          go back to the minimization routine.
         goto 111
```

```
241       endif
242
243 c           ---------- the end of the loop -------------
244
245 c     If task is neither FG nor NEW_X we terminate execution.
246
247       if (json_active) then
248          call json_write_aggregate(trim(lbfgsb_json), task, f,
249      +                             dsave(13))
250       endif
251
252       stop
253
254       end
255
256 c=======================  The end of driver2 ============================
257
```

4.4 driver2.f90

This driver shows how to replace the default stopping test by other termination criteria. It also illustrates how to print the values of several parameters during the course of the iteration. The sample problem used here is the same as in DRIVER1 (the extended Rosenbrock function with bounds on the variables). (Fortran-90 version)

```
1 !> \file driver2.f90
2
3 !
4 !  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 !  or "3-clause license")
6 !  Please read attached file License.txt
7 !
8 !
9 !                  DRIVER 2  in Fortran 90
10 !    ------------------------------------------------------------
11 !             CUSTOMIZED DRIVER FOR L-BFGS-B
12 !    ------------------------------------------------------------
13 !
14 !       L-BFGS-B is a code for solving large nonlinear optimization
15 !            problems with simple bounds on the variables.
16 !
17 !       The code can also be used for unconstrained problems and is
18 !       as efficient for these problems as the earlier limited memory
19 !                         code L-BFGS.
20 !
21 !       This driver illustrates how to control the termination of the
22 !       run and how to design customized output.
23 !
24 !    References:
25 !
26 !       [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 !       memory algorithm for bound constrained optimization",
28 !       SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 !
30 !       [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 !       Subroutines for Large Scale Bound Constrained Optimization"
32 !       Tech. Report, NAM-11, EECS Department, Northwestern University,
33 !       1994.
34 !
35 !
36 !         (Postscript files of these papers are available via anonymous
37 !          ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 !
39 !                               *  *  *
40 !
41 !         February 2011   (latest revision)
42 !         Optimization Center at Northwestern University
43 !         Instituto Tecnologico Autonomo de Mexico
44 !
45 !         Jorge Nocedal and Jose Luis Morales
46 !
47 !    **************
48       program driver
49       use json_writer
50
51 !     This driver shows how to replace the default stopping test
52 !     by other termination criteria. It also illustrates how to
53 !     print the values of several parameters during the course of
54 !     the iteration. The sample problem used here is the same as in
55 !     DRIVER1 (the extended Rosenbrock function with bounds on the
56 !     variables).
57
58       implicit none
```

```
59
60 !     Declare variables and parameters needed by the code.
61 !
62 !     Note that we suppress the default output (iprint = -1)
63 !     We suppress both code-supplied stopping tests because the
64 !     user is providing his/her own stopping criteria.
65
66       integer,  parameter    :: n = 25, m = 5, iprint = -1
67       integer,  parameter    :: dp = kind(1.0d0)
68       real(dp), parameter    :: factr  = 0.0d0, pgtol  = 0.0d0
69
70       character(len=60)      :: task, csave
71       logical                :: lsave(4)
72       integer                :: isave(44)
73       real(dp)               :: f
74       real(dp)               :: dsave(29)
75       integer,  allocatable  :: nbd(:), iwa(:)
76       real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)
77 !
78       real(dp)               :: t1, t2
79       integer                :: i
80
81 !     Test instrumentation: dump aggregate end-of-run state to JSON when
82 !     the environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
83 !     LBFGSB_NFG_LIMIT optionally overrides the nfg stopping threshold.
84       character(len=512)     :: lbfgsb_json
85       character(len=64)      :: env_nfg_limit
86       logical                :: json_active
87       integer                :: nfg_limit
88
89       allocate ( nbd(n), x(n), l(n), u(n), g(n) )
90       allocate ( iwa(3*n) )
91       allocate ( wa(2*m*n + 5*n + 11*m*m + 8*m) )
92 !
93 !    This driver shows how to replace the default stopping test
94 !    by other termination criteria. It also illustrates how to
95 !    print the values of several parameters during the course of
96 !    the iteration. The sample problem used here is the same as in
97 !    DRIVER1 (the extended Rosenbrock function with bounds on the
98 !    variables).
99 !    We now specify nbd which defines the bounds on the variables:
100 !                   l   specifies the lower bounds,
101 !                   u   specifies the upper bounds.
102
103 !    First set bounds on the odd numbered variables.
104
105       do 10 i=1, n,2
106          nbd(i)=2
107          l(i)=1.0d0
108          u(i)=1.0d2
109   10  continue
110
111 !    Next set bounds on the even numbered variables.
112
113       do 12 i=2, n,2
114          nbd(i)=2
115          l(i)=-1.0d2
116          u(i)=1.0d2
117   12   continue
118
119 !     We now define the starting point.
120
121       do 14 i=1, n
122          x(i)=3.0d0
123   14  continue
124
125 !     We now write the heading of the output.
126
127       write (6,16)
128   16  format(/,5x, 'Solving sample problem.', &
129              /,5x, ' (f = 0.0 at the optimal solution.)',/)
130
131
132 !     We start the iteration by initializing task.
133 !
134       task = 'START'
135
136 !     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
137       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
138       json_active = (len_trim(lbfgsb_json) > 0)
139       nfg_limit = 99
140       call get_environment_variable('LBFGSB_NFG_LIMIT', env_nfg_limit)
141       if (len_trim(env_nfg_limit) > 0) read(env_nfg_limit, *) nfg_limit
142
143 !        ------- the beginning of the loop ----------
144
145       do while( task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
```

```
146                task.eq.'START')
147
148 !     This is the call to the L-BFGS-B code.
149
150          call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint, &
151                      csave,lsave,isave,dsave)
152
153          if (task(1:2) .eq. 'FG') then
154
155 !        the minimization routine has returned to request the
156 !        function f and gradient g values at the current x.
157
158 !        Compute function value f for the sample problem.
159
160             f =.25d0*(x(1) - 1.d0)**2
161             do 20 i=2,n
162                f = f + (x(i) - x(i-1)**2)**2
163   20        continue
164             f = 4.d0*f
165
166 !        Compute gradient g for the sample problem.
167
168             t1 = x(2) - x(1)**2
169             g(1) = 2.d0*(x(1) - 1.d0) - 1.6d1*x(1)*t1
170             do 22 i= 2,n-1
171                t2 = t1
172                t1 = x(i+1) - x(i)**2
173                g(i) = 8.d0*t2 - 1.6d1*x(i)*t1
174   22        continue
175             g(n)=8.d0*t1
176 !
177       else
178 !
179          if (task(1:5) .eq. 'NEW_X') then
180 !
181 !       the minimization routine has returned with a new iterate.
182 !       At this point have the opportunity of stopping the iteration
183 !       or observing the values of certain parameters
184 !
185 !       First are two examples of stopping tests.
186
187 !       Note: task(1:4) must be assigned the value 'STOP' to terminate
188 !         the iteration and ensure that the final results are
189 !         printed in the default format. The rest of the character
190 !         string TASK may be used to store other information.
191
192 !       1) Terminate if the total number of f and g evaluations
193 !            exceeds 99.
194
195               if (isave(34) .ge. nfg_limit) &
196                  task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
197
198 !       2) Terminate if  |proj g|/(1+|f|) < 1.0d-10, where
199 !          "proj g" denoted the projected gradient
200
201               if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f))) &
202                 task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
203
204 !       We now wish to print the following information at each
205 !       iteration:
206 !
207 !         1) the current iteration number, isave(30),
208 !         2) the total number of f and g evaluations, isave(34),
209 !         3) the value of the objective function f,
210 !         4) the norm of the projected gradient,  dsve(13)
211 !
212 !       See the comments at the end of driver1 for a description
213 !       of the variables isave and dsave.
214
215             write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate' &
216                , isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
217
218 !        If the run is to be terminated, we print also the information
219 !        contained in task as well as the final value of x.
220
221             if (task(1:4) .eq. 'STOP') then
222                write (6,*) task
223                write (6,*) 'Final X='
224                write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
225             end if
226
227          end if
228       end if
229
230       end do
231 !           ---------- the end of the loop -------------
232
```

```
233 !     If task is neither FG nor NEW_X we terminate execution.
234
235       if (json_active) &
236          call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13))
237
238       end program driver
239
240 !======================= The end of driver2 ============================
241
```

4.5 driver3.f

This time-controlled driver shows that it is possible to terminate a run by elapsed CPU time, and yet be able to print all desired information. This driver also illustrates the use of two stopping criteria that may be used in conjunction with a limit on execution time. The sample problem used here is the same as in driver1 and driver2 (the extended Rosenbrock function with bounds on the variables). (Fortran-77 version)

```
1 c> \file driver3.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                           DRIVER 3 in Fortran 77
9 c     ------------------------------------------------------------
10 c           TIME-CONTROLLED DRIVER FOR L-BFGS-B (version 3.0)
11 c     ------------------------------------------------------------
12 c
13 c        L-BFGS-B is a code for solving large nonlinear optimization
14 c             problems with simple bounds on the variables.
15 c
16 c        The code can also be used for unconstrained problems and is
17 c        as efficient for these problems as the earlier limited memory
18 c                         code L-BFGS.
19 c
20 c        This driver shows how to terminate a run after some prescribed
21 c        CPU time has elapsed, and how to print the desired information
22 c        before exiting.
23 c
24 c     References:
25 c
26 c        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 c        memory algorithm for bound constrained optimization",
28 c        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 c
30 c        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 c        Subroutines for Large Scale Bound Constrained Optimization"
32 c        Tech. Report, NAM-11, EECS Department, Northwestern University,
33 c        1994.
34 c
35 c
36 c          (Postscript files of these papers are available via anonymous
37 c           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 c
39 c                               *  *  *
40 c
41 c         February 2011   (latest revision)
42 c         Optimization Center at Northwestern University
43 c         Instituto Tecnologico Autonomo de Mexico
44 c
45 c         Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c         L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c         Optimization"  (2011). To appear in  ACM Transactions on
48 c         Mathematical Software,
49 c
50 c
51 c     **************
52
53       program driver
54       use json_writer
55
56 c     This time-controlled driver shows that it is possible to terminate
57 c     a run by elapsed CPU time, and yet be able to print all desired
58 c     information. This driver also illustrates the use of two
59 c     stopping criteria that may be used in conjunction with a limit
60 c     on execution time. The sample problem used here is the same as in
61 c     driver1 and driver2 (the extended Rosenbrock function with bounds
62 c     on the variables).
63
64       integer          nmax, mmax
65       parameter(nmax=1024,mmax=17)
```

```
66 c        nmax is the dimension of the largest problem to be solved.
67 c        mmax is the maximum number of limited memory corrections.
68
69 c     Declare the variables needed by the code.
70 c       A description of all these variables is given at the end of
71 c       driver1.
72
73       character*60     task, csave
74       logical          lsave(4)
75       integer          n, m, iprint,
76      +                 nbd(nmax), iwa(3*nmax), isave(44)
77       double precision f, factr, pgtol,
78      +                 x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
79      +                 wa(2*mmax*nmax+5*nmax+11*mmax*mmax+8*mmax)
80
81 c     Declare a few additional variables for the sample problem
82 c       and for keeping track of time.
83
84       double precision t1, t2, time1, time2, tlimit
85       integer          i, j
86
87 c     Test instrumentation: dump per-iteration state to JSON when the
88 c     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
89 c     LBFGSB_TLIMIT optionally overrides tlimit for reproducibility.
90       character*512    lbfgsb_json
91       character*64     env_tlimit
92       logical          json_active
93
94 c     We specify a limite on the CPU time (in seconds).
95
96       tlimit = 0.2
97       call get_environment_variable('LBFGSB_TLIMIT', env_tlimit)
98       if (len_trim(env_tlimit) .gt. 0) read(env_tlimit, *) tlimit
99
100 c     We suppress the default output.  (The user could also elect to
101 c       use the default output by choosing iprint >= 0.)
102
103       iprint = -1
104
105 c     We suppress the code-supplied stopping tests because we will
106 c       provide our own termination conditions
107
108       factr=0.0d0
109       pgtol=0.0d0
110
111 c     We specify the dimension n of the sample problem and the number
112 c        m of limited memory corrections stored.  (n and m should not
113 c        exceed the limits nmax and mmax respectively.)
114
115       n=1000
116       m=10
117
118 c     We now specify nbd which defines the bounds on the variables:
119 c                     l   specifies the lower bounds,
120 c                     u   specifies the upper bounds.
121
122 c     First set bounds on the odd-numbered variables.
123
124       do 10 i=1,n,2
125          nbd(i)=2
126          l(i)=1.0d0
127          u(i)=1.0d2
128    10  continue
129
130 c     Next set bounds on the even-numbered variables.
131
132       do 12 i=2,n,2
133          nbd(i)=2
134          l(i)=-1.0d2
135          u(i)=1.0d2
136    12   continue
137
138 c     We now define the starting point.
139
140       do 14 i=1,n
141          x(i)=3.0d0
142    14  continue
143
144 c     We now write the heading of the output.
145
146       write (6,16)
147    16  format(/,5x, 'Solving sample problem.',
148      +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
149
150 c     We start the iteration by initializing task.
151 c
152       task = 'START'
```

```
153
154 c     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
155       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
156       json_active = (len_trim(lbfgsb_json) .gt. 0)
157
158 c        ------- the beginning of the loop ----------
159
160 c     We begin counting the CPU time.
161
162       call timer(time1)
163
164  111  continue
165
166 c     This is the call to the L-BFGS-B code.
167
168       call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
169      +            csave,lsave,isave,dsave)
170
171       if (task(1:2) .eq. 'FG') then
172 c        the minimization routine has returned to request the
173 c        function f and gradient g values at the current x.
174 c        Before evaluating f and g we check the CPU time spent.
175
176          call timer(time2)
177          if (time2-time1 .gt. tlimit) then
178             task='STOP: CPU EXCEEDING THE TIME LIMIT.'
179
180 c          Note: Assigning task(1:4)='STOP' will terminate the run;
181 c          setting task(7:9)='CPU' will restore the information at
182 c          the latest iterate generated by the code so that it can
183 c          be correctly printed by the driver.
184
185 c          In this driver we have chosen to disable the
186 c          printing options of the code (we set iprint=-1);
187 c          instead we are using customized output: we print the
188 c          latest value of x, the corresponding function value f and
189 c          the norm of the projected gradient |proj g|.
190
191 c          We print out the information contained in task.
192
193             write (6,*) task
194
195 c          We print the latest iterate contained in wa(j+1:j+n), where
196 c
197             j = 3*n+2*m*n+11*m**2
198             write (6,*) 'Latest iterate X ='
199             write (6,'((1x,1p, 6(1x,d11.4)))') (wa(i),i = j+1,j+n)
200
201 c          We print the function value f and the norm of the projected
202 c          gradient |proj g| at the last iterate; they are stored in
203 c          dsave(2) and dsave(13) respectively.
204
205             write (6,'(a,1p,d12.5,4x,a,1p,d12.5)')
206      +         'At latest iterate   f =',dsave(2),'|proj g| =',dsave(13)
207
208          else
209
210 c          The time limit has not been reached and we compute
211 c          the function value f for the sample problem.
212
213             f=.25d0*(x(1)-1.d0)**2
214             do 20 i=2,n
215                f=f+(x(i)-x(i-1)**2)**2
216   20        continue
217             f=4.d0*f
218
219 c          Compute gradient g for the sample problem.
220
221             t1=x(2)-x(1)**2
222             g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
223             do 22 i=2,n-1
224                t2=t1
225                t1=x(i+1)-x(i)**2
226                g(i)=8.d0*t2-1.6d1*x(i)*t1
227   22        continue
228             g(n)=8.d0*t1
229
230          endif
231
232 c          go back to the minimization routine.
233          goto 111
234       endif
235 c
236       if (task(1:5) .eq. 'NEW_X') then
237 c        the minimization routine has returned with a new iterate.
238 c        The time limit has not been reached, and we test whether
239 c        the following two stopping tests are satisfied:
```

```
240
241 c        1) Terminate if the total number of f and g evaluations
242 c             exceeds 900.
243
244          if (isave(34) .ge. 900)
245      +      task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
246
247 c        2) Terminate if  |proj g|/(1+|f|) < 1.0d-10.
248
249          if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f)))
250      +      task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
251
252 c        We wish to print the following information at each iteration:
253 c          1) the current iteration number, isave(30),
254 c          2) the total number of f and g evaluations, isave(34),
255 c          3) the value of the objective function f,
256 c          4) the norm of the projected gradient,  dsve(13)
257 c
258 c        See the comments at the end of driver1 for a description
259 c        of the variables isave and dsave.
260
261
262          write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate'
263      +      ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
264
265 c        If the run is to be terminated, we print also the information
266 c        contained in task as well as the final value of x.
267
268
269          if (task(1:4) .eq. 'STOP') then
270             write (6,*) task
271             write (6,*) 'Final X='
272             write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
273          endif
274
275 c          go back to the minimization routine.
276          goto 111
277
278       endif
279
280 c           ---------- the end of the loop -------------
281
282 c     If task is neither FG nor NEW_X we terminate execution.
283
284       if (json_active) then
285          call json_write_aggregate(trim(lbfgsb_json), task, f,
286      +                             dsave(13))
287       endif
288
289       stop
290
291       end
292
293 c======================= The end of driver3 ============================
294
```

4.6 driver3.f90

This time-controlled driver shows that it is possible to terminate a run by elapsed CPU time, and yet be able to print all desired information. This driver also illustrates the use of two stopping criteria that may be used in conjunction with a limit on execution time. The sample problem used here is the same as in driver1 and driver2 (the extended Rosenbrock function with bounds on the variables). (Fortran-90 version)

```
1 !> \file driver3.f90
2
3 !
4 !  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 !  or "3-clause license")
6 !  Please read attached file License.txt
7 !
8 !                           DRIVER 3  in Fortran 90
9 !     ----------------------------------------------------------
10 !           TIME-CONTROLLED DRIVER FOR L-BFGS-B
11 !     ----------------------------------------------------------
12 !
13 !        L-BFGS-B is a code for solving large nonlinear optimization
14 !             problems with simple bounds on the variables.
15 !
16 !        The code can also be used for unconstrained problems and is
17 !        as efficient for these problems as the earlier limited memory
18 !                         code L-BFGS.
19 !
```

```
20 !        This driver shows how to terminate a run after some prescribed
21 !        CPU time has elapsed, and how to print the desired information
22 !        before exiting.
23 !
24 !     References:
25 !
26 !        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 !        memory algorithm for bound constrained optimization",
28 !        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 !
30 !        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 !        Subroutines for Large Scale Bound Constrained Optimization"
32 !        Tech. Report, NAM-11, EECS Department, Northwestern University,
33 !        1994.
34 !
35 !
36 !          (Postscript files of these papers are available via anonymous
37 !           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 !
39 !                               *  *  *
40 !
41 !          February 2011   (latest revision)
42 !          Optimization Center at Northwestern University
43 !          Instituto Tecnologico Autonomo de Mexico
44 !
45 !          Jorge Nocedal and Jose Luis Morales
46 !
47 !     **************
48
49       program driver
50       use json_writer
51
52 !     This time-controlled driver shows that it is possible to terminate
53 !     a run by elapsed CPU time, and yet be able to print all desired
54 !     information. This driver also illustrates the use of two
55 !     stopping criteria that may be used in conjunction with a limit
56 !     on execution time. The sample problem used here is the same as in
57 !     driver1 and driver2 (the extended Rosenbrock function with bounds
58 !     on the variables).
59
60       implicit none
61
62 !     We specify a limit on the CPU time (tlimit = 10 seconds)
63 !
64 !     We suppress the default output (iprint = -1). The user could
65 !       also elect to use the default output by choosing iprint >= 0.)
66 !     We suppress the code-supplied stopping tests because we will
67 !       provide our own termination conditions
68 !     We specify the dimension n of the sample problem and the number
69 !        m of limited memory corrections stored.
70
71       integer,  parameter    :: n = 1000, m = 10, iprint = -1
72       integer,  parameter    :: dp = kind(1.0d0)
73       real(dp), parameter    :: factr  = 0.0d0, pgtol  = 0.0d0
74       real(dp)               :: tlimit
75 !
76       character(len=60)      :: task, csave
77       logical                :: lsave(4)
78       integer                :: isave(44)
79       real(dp)               :: f
80       real(dp)               :: dsave(29)
81       integer,  allocatable  :: nbd(:), iwa(:)
82       real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)
83 !
84       real(dp)               :: t1, t2, time1, time2
85       integer                :: i, j
86
87 !     Test instrumentation: dump per-iteration state to JSON when the
88 !     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
89 !     LBFGSB_TLIMIT optionally overrides tlimit for reproducibility.
90       character(len=512)     :: lbfgsb_json
91       character(len=64)      :: env_tlimit
92       logical                :: json_active
93
94       allocate ( nbd(n), x(n), l(n), u(n), g(n) )
95       allocate ( iwa(3*n) )
96       allocate ( wa(2*m*n + 5*n + 11*m*m + 8*m) )
97
98 !     This time-controlled driver shows that it is possible to terminate
99 !     a run by elapsed CPU time, and yet be able to print all desired
100 !     information. This driver also illustrates the use of two
101 !     stopping criteria that may be used in conjunction with a limit
102 !     on execution time. The sample problem used here is the same as in
103 !     driver1 and driver2 (the extended Rosenbrock function with bounds
104 !     on the variables).
105
106 !     We now specify nbd which defines the bounds on the variables:
```

```
107 !                    l   specifies the lower bounds,
108 !                    u   specifies the upper bounds.
109
110 !     First set bounds on the odd-numbered variables.
111
112       do 10 i=1, n,2
113          nbd(i)=2
114          l(i)=1.0d0
115          u(i)=1.0d2
116    10 continue
117
118 !     Next set bounds on the even-numbered variables.
119
120       do 12 i=2, n,2
121          nbd(i)=2
122          l(i)=-1.0d2
123          u(i)=1.0d2
124    12  continue
125
126 !     We now define the starting point.
127
128       do 14 i=1, n
129          x(i)=3.0d0
130    14 continue
131
132 !     We now write the heading of the output.
133
134       write (6,16)
135    16 format(/,5x, 'Solving sample problem.',&
136              /,5x, ' (f = 0.0 at the optimal solution.)',/)
137
138 !     We start the iteration by initializing task.
139
140       task = 'START'
141
142 !     Initialise tlimit (with optional override from LBFGSB_TLIMIT) and
143 !     enable JSON output if LBFGSB_JSON_OUTPUT is set.
144       tlimit = 10.0d0
145       call get_environment_variable('LBFGSB_TLIMIT', env_tlimit)
146       if (len_trim(env_tlimit) > 0) read(env_tlimit, *) tlimit
147       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
148       json_active = (len_trim(lbfgsb_json) > 0)
149
150 !        ------- the beginning of the loop ----------
151
152 !     We begin counting the CPU time.
153
154       call timer(time1)
155
156       do while( task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
157                 task.eq.'START')
158
159 !     This is the call to the L-BFGS-B code.
160
161          call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa, &
162                      task,iprint, csave,lsave,isave,dsave)
163
164          if (task(1:2) .eq. 'FG') then
165
166 !        the minimization routine has returned to request the
167 !        function f and gradient g values at the current x.
168 !        Before evaluating f and g we check the CPU time spent.
169
170          call timer(time2)
171          if (time2-time1 .gt. tlimit) then
172             task='STOP: CPU EXCEEDING THE TIME LIMIT.'
173
174 !          Note: Assigning task(1:4)='STOP' will terminate the run;
175 !          setting task(7:9)='CPU' will restore the information at
176 !          the latest iterate generated by the code so that it can
177 !          be correctly printed by the driver.
178
179 !          In this driver we have chosen to disable the
180 !          printing options of the code (we set iprint=-1);
181 !          instead we are using customized output: we print the
182 !          latest value of x, the corresponding function value f and
183 !          the norm of the projected gradient |proj g|.
184
185 !          We print out the information contained in task.
186
187             write (6,*) task
188
189 !          We print the latest iterate contained in wa(j+1:j+n), where
190
191             j = 3*n+2*m*n+11*m**2
192             write (6,*) 'Latest iterate X ='
193             write (6,'((1x,1p, 6(1x,d11.4)))') (wa(i),i = j+1,j+n)
```

```
194
195 !          We print the function value f and the norm of the projected
196 !          gradient |proj g| at the last iterate; they are stored in
197 !          dsave(2) and dsave(13) respectively.
198
199              write (6,'(a,1p,d12.5,4x,a,1p,d12.5)') &
200              'At latest iterate   f =',dsave(2),'|proj g| =',dsave(13)
201            else
202
203 !          The time limit has not been reached and we compute
204 !          the function value f for the sample problem.
205
206              f=.25d0*(x(1)-1.d0)**2
207              do 20 i=2, n
208                 f=f+(x(i)-x(i-1)**2)**2
209   20         continue
210              f=4.d0*f
211
212 !          Compute gradient g for the sample problem.
213
214               t1 = x(2) - x(1)**2
215               g(1) = 2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
216               do 22 i=2,n-1
217                  t2=t1
218                  t1=x(i+1)-x(i)**2
219                  g(i)=8.d0*t2-1.6d1*x(i)*t1
220   22          continue
221               g(n)=8.d0*t1
222            endif
223
224 !          go back to the minimization routine.
225          else
226
227            if (task(1:5) .eq. 'NEW_X') then
228
229 !        the minimization routine has returned with a new iterate.
230 !        The time limit has not been reached, and we test whether
231 !        the following two stopping tests are satisfied:
232
233 !        1) Terminate if the total number of f and g evaluations
234 !             exceeds 900.
235
236              if (isave(34) .ge. 900) &
237              task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
238
239 !        2) Terminate if  |proj g|/(1+|f|) < 1.0d-10.
240
241              if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f))) &
242              task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
243
244 !        We wish to print the following information at each iteration:
245 !          1) the current iteration number, isave(30),
246 !          2) the total number of f and g evaluations, isave(34),
247 !          3) the value of the objective function f,
248 !          4) the norm of the projected gradient,  dsve(13)
249 !
250 !        See the comments at the end of driver1 for a description
251 !        of the variables isave and dsave.
252
253              write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate' &
254              ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
255
256 !        If the run is to be terminated, we print also the information
257 !        contained in task as well as the final value of x.
258
259              if (task(1:4) .eq. 'STOP') then
260                 write (6,*) task
261                 write (6,*) 'Final X='
262                 write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
263              endif
264
265            endif
266          end if
267        end do
268
269 !      If task is neither FG nor NEW_X we terminate execution.
270
271        if (json_active) &
272           call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13))
273
274        end program driver
275
276 !======================= The end of driver3 ============================
277
```

Index

active
    active.f, 4
active.f
    active, 4

bmv
    bmv.f, 5
bmv.f
    bmv, 5

cauchy
    cauchy.f, 6
cauchy.f
    cauchy, 6
cmprlb
    cmprlb.f, 10
cmprlb.f
    cmprlb, 10

dcsrch
    dcsrch.f, 11
dcsrch.f
    dcsrch, 11
dcstep
    dcstep.f, 14
dcstep.f
    dcstep, 14

errclb
    errclb.f, 15
errclb.f
    errclb, 15

formk
    formk.f, 16
formk.f
    formk, 16
formt
    formt.f, 18
formt.f
    formt, 18
freev
    freev.f, 19
freev.f
    freev, 19

hpsolb
    hpsolb.f, 20
hpsolb.f
    hpsolb, 20

lnsrlb
    lnsrlb.f, 21
lnsrlb.f
    lnsrlb, 21

mainlb
    mainlb.f, 24
mainlb.f
    mainlb, 24
matupd
    matupd.f, 28
matupd.f
    matupd, 28

prn1lb
    prn1lb.f, 29
prn1lb.f
    prn1lb, 29
prn2lb
    prn2lb.f, 31
prn2lb.f
    prn2lb, 31
prn3lb
    prn3lb.f, 32
prn3lb.f
    prn3lb, 32
projgr
    projgr.f, 34
projgr.f
    projgr, 34

setulb
    setulb.f, 35
setulb.f
    setulb, 35
src/active.f, 4
src/bmv.f, 5
src/cauchy.f, 6
src/cmprlb.f, 10
src/dcsrch.f, 11
src/dcstep.f, 13
src/errclb.f, 15
src/formk.f, 16
src/formt.f, 18
src/freev.f, 19
src/hpsolb.f, 20
src/lnsrlb.f, 21
src/mainlb.f, 24
src/matupd.f, 28
src/prn1lb.f, 29
src/prn2lb.f, 31
src/prn3lb.f, 32
src/projgr.f, 34
src/setulb.f, 35
src/subsm.f, 39
src/timer.f, 42
subsm
    subsm.f, 39
subsm.f
    subsm, 39

timer

timer.f, 42
timer.f
timer, 42